

Multi-Line Insurance Policy and Claims Management System

College name: Alagappa Chettiar Government College of Engineering & Technology, Karaikudi

College code: 9176

Team Member	Email ID	NM ID
Ashwini D – Team Leader	ashwimeenu131@gmail.com	E880FD940A213AAB5BA44C6D0042742A
Priyadarshini R	ravipriyadarshini1605@gmail.com	A0F81BFA17568919B594DDC8856A78B6
Manjula V	manjulavasumalai2006@gmail.com	73B5F8985E61FE994CA4408532217E4C
Alageshwari M	alageshwari524555@gamil.com	EF4823F7C6F1980855048B4D06C52372
Rohit Naren K	rohitnaren2006@gmail.com	FE79B8133F899CA89BB442F

Executive Summary & Abstract

The Multi-Line Insurance Policy and Claims Management System represents a state-of-the-art, enterprise-grade Salesforce implementation engineered to overhaul traditional, fragmented insurance workflows. By consolidating Vehicle, Property, and Life insurance portfolios into a single, unified database schema, this project addresses systemic legacy challenges such as manual underwriting bottlenecks, inconsistent rating matrices, delayed claims routing, and lack of real-time operational visibility. Leveraging declarative tools like Screen Flows, Record-Triggered Automation, Approval Processes, and programmatic capabilities including custom Apex controllers, invocable methods, and Lightning Web Components (LWC), the platform ensures end-to-end data integrity, robust security enforcement via profile-based sharing models, and intuitive user experiences. This documentation provides an exhaustive technical blueprint, architectural mapping, project scheduling metrics, and comprehensive code repositories detailing every phase of the project's lifecycle.

1. INTRODUCTION

1.1 Project Overview:

The Multi-Line Insurance Policy and Claims Management System is an enterprise-class Salesforce solution designed to centralize and streamline the lifecycle management of Vehicle, Property, and Life Insurance portfolios. Historically, insurance institutions operated on siloed legacy systems that required manual data re-entry, disparate spreadsheet tracking for claims approvals, and disconnected communication channels between underwriting teams, claims adjusters, and management executives. This project replaces these fragmented processes with a cohesive architecture built upon custom Salesforce objects, specialized Record Types, dynamic Field Sets, guided Screen Flows, robust Apex automation routines, multi-tier Approval Processes, and responsive Lightning Web Components.

The solution empowers insurance agents to originate and govern policies through a flexible data framework. Guided Screen Flows step users through data capture requirements, while data validation rules enforce strict business constraints (such as standard 17-character VIN verification for automotive assets). Furthermore, custom Apex calculation classes evaluate regional risk factors and model years to compute precise premiums instantaneously. When an adverse incident occurs, intake logging automatically triggers record-triggered flow automation that identifies the corresponding policy line and routes the claim to the appropriate support queue.

For claims adjusters, an interactive LWC dashboard provides a consolidated command center displaying open case queues, compensation amounts, policyholders, aging intervals, and real-time processing statuses. High-value cases surpassing configured monetary ceilings enter an automated, multi-level approval pipeline engaging Senior Adjusters and Department Managers. Role-specific Permission Sets and state-based sharing rules safeguard sensitive client data, ensuring compliance and operational security across all organizational tiers.

1.2 Purpose:

The overarching objective of the Multi-Line Insurance Policy and Claims Management System is to provide a scalable, secure, and unified platform that eliminates operational inefficiencies inherent in traditional insurance administration. Specifically, the project targets the following strategic outcomes:

- **Standardization:** Establish uniform quoting and issuance protocols across diverse product lines (Vehicle, Property, and Life).
- **Automation:** Minimize manual clerical tasks by automating premium calculations, claim routing, and escalation workflows.
- **Visibility:** Deliver a real-time, ³⁶⁰-degree view of customer profiles, active policy contracts, and pending claim files to empower decision-makers.
- **Accuracy:** Enforce strict data hygiene through automated field validations and structural constraints.
- **Agility:** Utilize modular, low-code and pro-code components to facilitate future product expansions and regulatory adaptations without requiring ground-up system re-engineering.

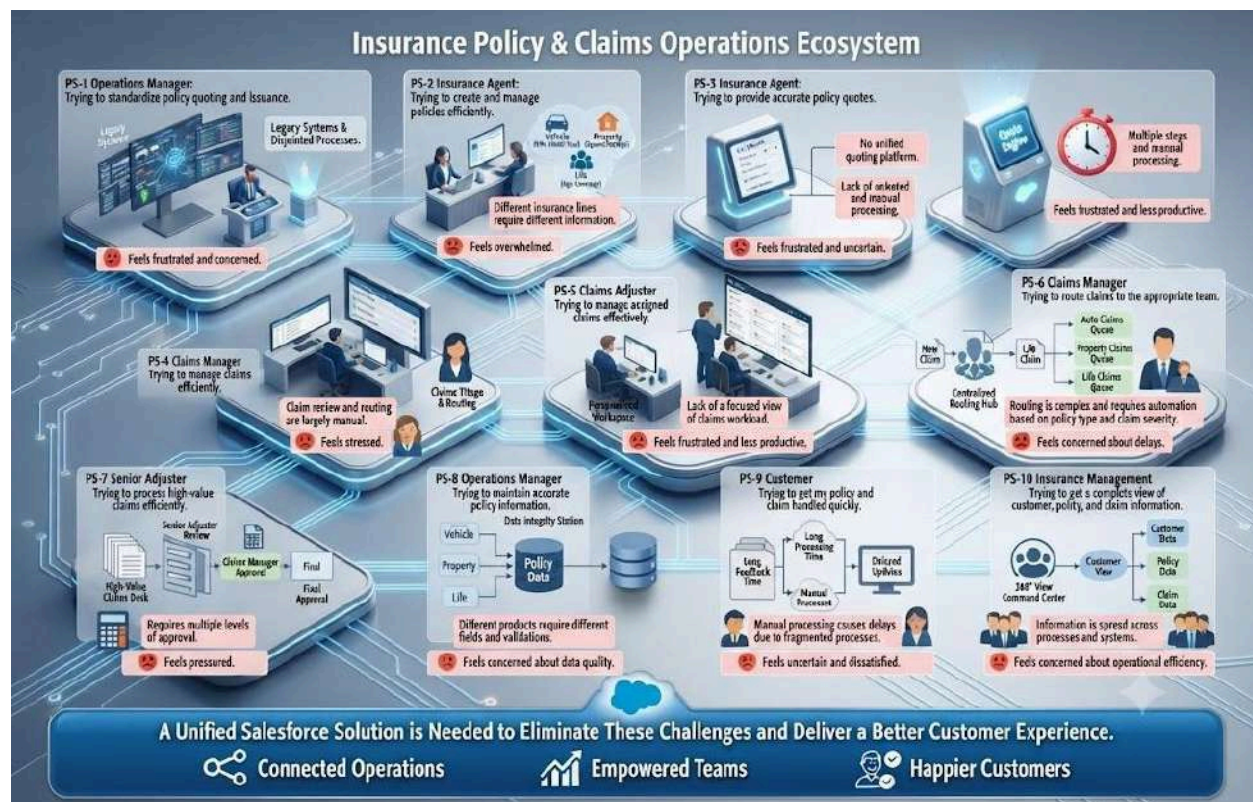
2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS ID	User Role	Intended Goal	Existing Obstacle	Underlying Root Cause	Emotional Impact
PS-1	Insurance Operations Manager	Standardize policy quoting and issuance	The procedure remains manual, sluggish, and irregular	Legacy architectures and processes are heavily fragmented	Frustrated and apprehensive
PS-2	Insurance Agent	Create and update policies seamlessly	Distinct insurance lines demand varying data inputs	Vehicle, Property, and Life portfolios have diverse information criteria	Overwhelmed
PS-3	Insurance Agent	Deliver precise policy estimates	Quoting requires multi-step manual handling	There is no unified quoting engine	Frustrated and uncertain
PS-4	Claims Manager	Manage incident files effectively	Claim inspection and routing are predominantly manual	Dispute workflows lack centralization and automation	Stressed
PS-5	Claims Adjuster	Handle assigned	Staff lack a consolidated	Incident details are	Frustrated and

		claims productively	ed outlook of individual workloads	scattered across disparate modules	disengage d
PS-6	Claims Manager	Direct cases to appropriat e groups	Tickets must be categorize d by policy type and severity	Routing rules are intricate and demand automatio n	Concerne d regarding delays
PS-7	Senior Adjuster	Process high-value claims efficiently	Authorizati on necessitat es multiple layers of scrutiny	High-value payouts require formal escalation tracking	Pressured
PS-8	Insurance Operation s Manager	Preserve policy data accuracy	Different products require specialized fields and checks	Asset lines involve inconsiste nt validation needs	Anxious about data hygiene
PS-9	End Customer	Expedite policy and claim services	Manual delays hinder fast turnaroun d times	Service pipelines are disjointed	Uncertain and dissatisfie d
PS-10	Insurance Managem ent	Gain total oversight of client, policy, and claim	Informatio n is siloed across separate environme	The organizati on lacks a centralized database	Worried about operationa l output

		metrics	nts		
--	--	---------	-----	--	--

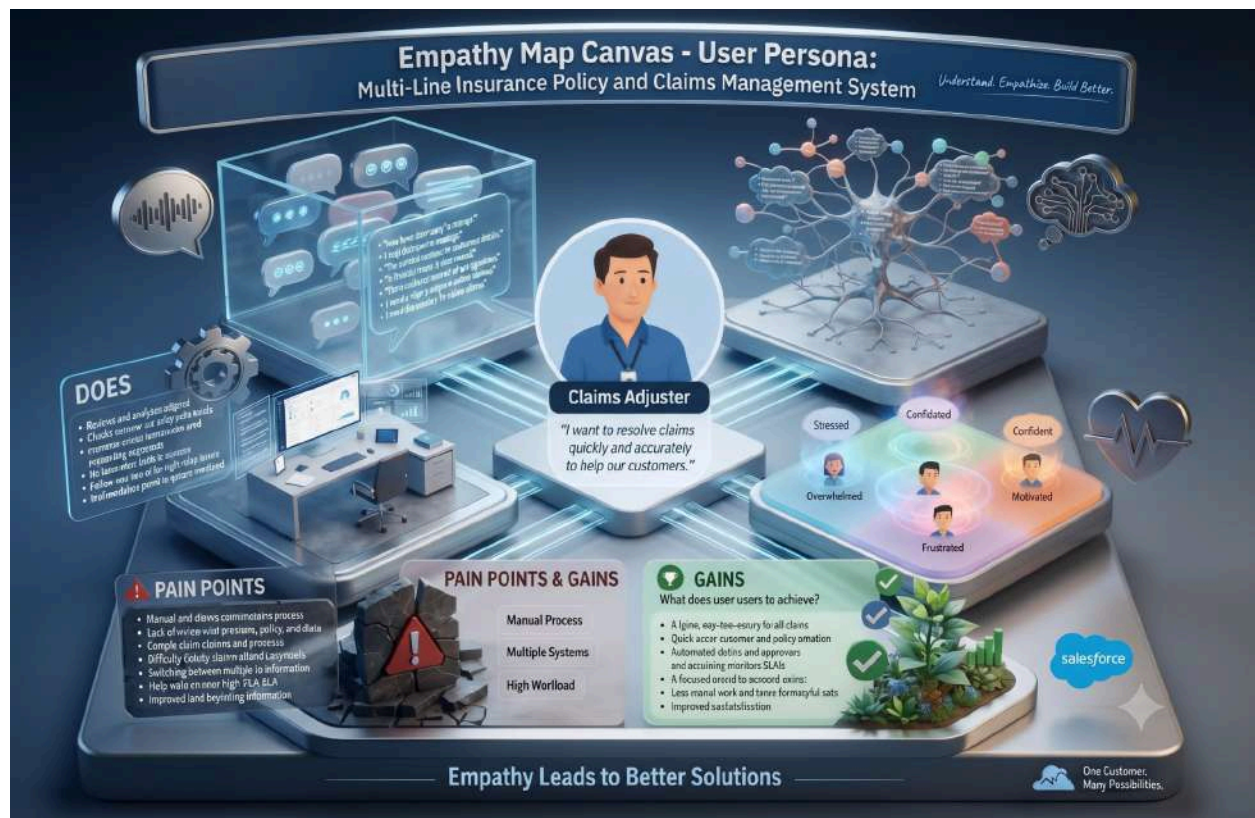


2.2 Empathy Map Canvas

Target User Persona: Insurance Claims Adjuster

- **Think & Feel:** Highly concerned about meeting strict Service Level Agreements (SLAs), resolving customer claims quickly, and managing high caseload volumes without missing critical details.
- **Hear:** Frequent inquiries from policyholders regarding payout statuses, team discussions on workload bottlenecks, and management reminders regarding compliance metrics.
- **See:** Cluttered desktop environments, multiple browser tabs containing unlinked policy spreadsheets, email threads regarding claim evidence, and unorganized claim queues.
- **Say & Do:** Articulates frustration over switching between disparate systems; manually reviews loss reports, communicates with claimants, updates status fields, and escalates complex files.

- **Pain Points:** Time-consuming manual reviews, absence of a unified data view, complex routing delays, and high cognitive load from managing multiple insurance product requirements simultaneously.
- **Gains:** A centralized LWC claims dashboard, automated queue routing, real-time status tracking, instant access to policyholder history, and streamlined approval execution.



2.3 Brainstorming & Idea Prioritization

Alternative Concepts Explored:

1. **Manual Paper Registers:** Traditional ledger-based record keeping. Rejected due to extreme error rates, lack of remote access, and impossible scaling.
2. **Spreadsheet Tracking Systems:** Utilizing local Excel files shared via network drives. Rejected due to version control conflicts, poor security governance, and lack of real-time multi-user concurrency.
3. **Standalone Custom Web Portal:** Building a bespoke application from scratch using Node.js and React. Rejected due to exorbitant development costs, lengthy

time-to-market, and lack of out-of-the-box enterprise security and workflow engines.

4. **Salesforce CRM Platform:** Utilizing the Salesforce multi-tenant cloud ecosystem. Selected as the optimal solution due to built-in relational data mapping, declarative automation tools, robust security frameworks, and seamless extensibility via Lightning Web Components.



3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Key Stages:

1. **Awareness & Inquiry:** Prospective clients explore multi-line insurance options through agency channels.
2. **Product Selection:** Clients evaluate Vehicle, Property, or Life coverage options based on personal asset profiles.
3. **Quote Request:** Agents initiate interactive guided flows to collect risk parameters.
4. **Data Capture & Validation:** The system captures product-specific fields and enforces strict validation checks (e.g., 17-character VIN verification).

5. **Premium Calculation:** Apex rating logic evaluates risk parameters and computes localized premiums.
6. **Policy Issuance:** Draft policies are converted into active contracts stored within custom Salesforce objects.
7. **Incident Initiation:** Policyholders report losses, generating formal claim entries linked to parent policies.
8. **Automated Routing:** Record-triggered flows categorize and assign claims to specialized adjuster queues.
9. **Review & Authorization:** Adjusters inspect evidence; high-value claims trigger multi-tier management approvals.
10. **Resolution & Payout:** Claims are formally approved or rejected, updating statuses and notifying clients.
11. **Reporting & Analytics:** Management monitors operational KPIs via centralized dashboards and reports.



3.2 Solution Requirements

Functional Requirements Matrix

ID	Functional Specification Description	Priority
FR-01	The application shall allow personnel to	High

	generate and manage policy entries for Vehicle, Property, and Life categories.	
FR-02	The system shall support distinct Record Types for Vehicle, Property, and Life insurance variants.	High
FR-03	The platform shall capture product-specific data points such as VIN for autos and square footage/construction year for properties.	High
FR-04	The solution shall feature a guided Auto Quoting Screen Flow to collect holder names, start dates, states, VINs, and model years.	High
FR-05	The application shall automatically instantiate a draft policy record during the quoting workflow.	High
FR-06	The system shall compute policy premiums utilizing custom Apex calculation routines.	High
FR-07	The platform shall validate vehicle	High

	identification numbers to guarantee that every VIN consists of precisely 17 characters.	
FR-08	The system shall enable users to establish and oversee claims connected to active policies.	High
FR-09	The solution shall support specialized claim Record Types for Accident, Property damage, and Life incidents.	High
FR-10	The system shall automatically route claims to appropriate queues based on the parent policy category.	High
FR-11	The application shall detect high-value claims according to predefined financial thresholds.	High
FR-12	The platform shall route qualifying high-value claims through a sequential multi-stage approval workflow involving Senior Adjusters and Managers.	High
FR-13	The system shall provide review capabilities	High

	allowing approvers to approve, reject, and add explanatory comments.	
FR-14	The solution shall provide claims adjusters with an interactive LWC-based dashboard displaying active assignments.	High
FR-15	The dashboard shall exhibit critical metrics including Claim IDs, Policy Types, Holder Names, Claim Amounts, Statuses, and Days Open.	High
FR-16	The dashboard shall allow users to filter records dynamically by All, Auto, Property, and Life types.	Medium
FR-17	The application shall execute automated record updates via declarative Record-Triggered Flows.	High
FR-18	The platform shall support region-based record visibility and sharing rules for authorized insurance users.	Medium
FR-19	The system shall enforce role-specific access permissions via targeted	High

	Permission Sets for Agents, Adjusters, and Managers.	
FR-20	The application shall provide comprehensive reporting and dashboard tools for monitoring client, contract, and claim metrics.	Medium

Non-Functional Requirements Matrix

ID	Category	Description
NFR-01	Usability	The platform must provide streamlined, wizard-style Screen Flows that reduce clicks and simplify quoting for agents.
NFR-02	Performance	Quoting actions, rating calculations, routing flows, and dashboard queries must execute rapidly to ensure smooth operations.
NFR-03	Scalability	The architecture must support current vehicle, property, and life lines while remaining open to future product additions.
NFR-04	Data Integrity	Information inputs must

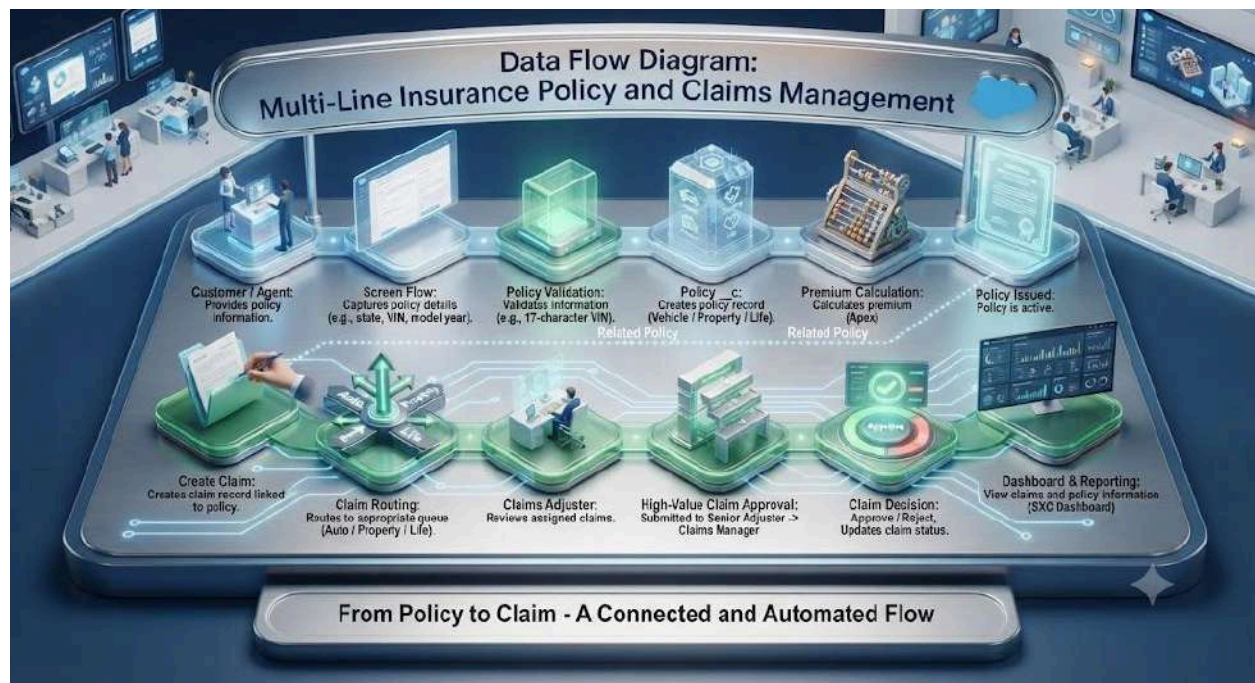
		undergo rigorous validation checks to preserve consistency across portfolios.
NFR-05	Security	System access must be strictly managed via profile-based assignments, sharing criteria, and Permission Sets.
NFR-06	Maintainability	The solution should utilize modular components like Flows, Apex handlers, and LWCs to allow independent maintenance.
NFR-07	Reliability	Routing rules, financial calculations, and approval sequences must execute consistently based on configuration rules.
NFR-08	Extensibility	The data model must accommodate specialized product fields without requiring entirely separate database tables.
NFR-09	Visibility	Staff must enjoy prompt access to operational metrics through centralized layouts and interactive dashboards.

NFR-10	User Adoption	The user interface must offer an intuitive layout that minimizes manual labor and encourages rapid staff onboarding.
--------	---------------	--

3.3 Data Flow Diagram

Process Sequence:

Client/Agent Input → Interactive Screen Flow → Validation Engine → Custom Policy Object (Policy__c) → Apex Rating Engine (PremiumCalculator) → Claim Registration (Claim__c) → Record-Triggered Routing Flow → Specialized Support Queue → Adjuster Workspace (LWC) → Multi-Tier Approval Workflow → Status Commitment → Management Dashboard & Reporting Suite



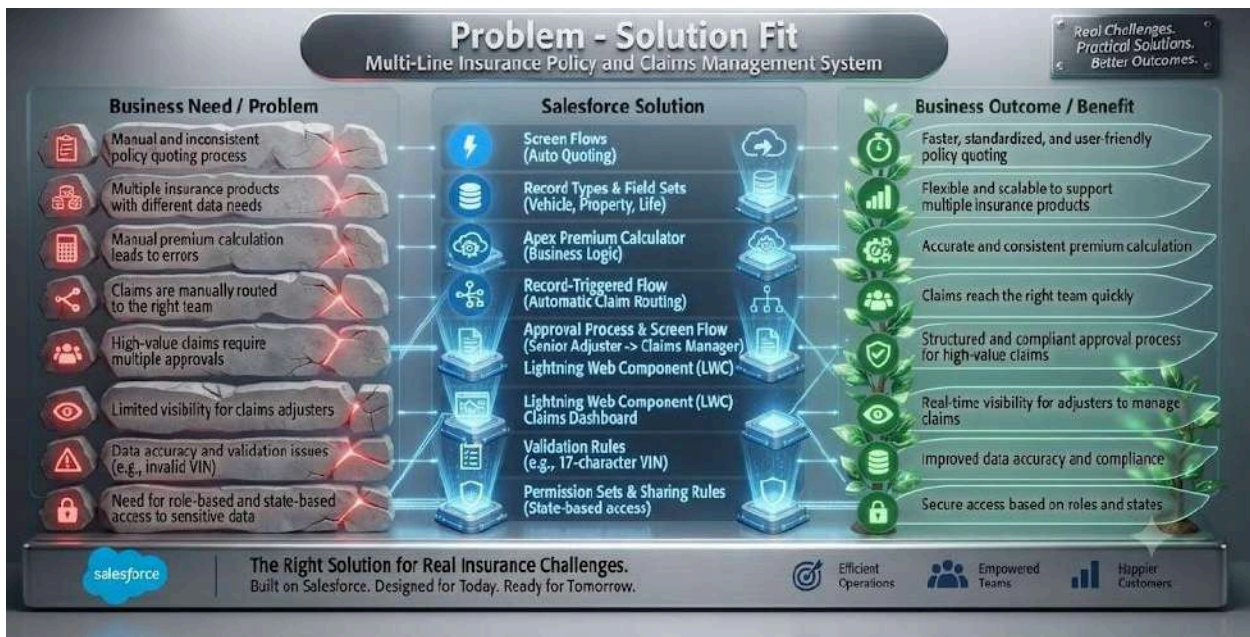
3.4 Technology Stack

Layer Component	Implementation Technology
User Interface Layer	Salesforce Lightning Experience, Lightning Web Components (LWC), SLDS (Salesforce Lightning Design System)
Business Logic Layer	Salesforce Screen Flows, Record-Triggered Flows, Apex Classes, Apex Triggers
Database & Storage	Custom Objects (Policy__c, Claim__c), Standard Objects (Contact, User, Queue)
Automation Engine	Declarative Flows, Validation Rules, Approval Processes, Invocable Actions
Calculation Services	Apex Premium Calculator, Custom Apex Controllers with Wrapper Classes
Data Configuration	Record Types, Field Sets, Custom Picklists, Lookup Relationships
Security Architecture	Profile Assignments, Permission Sets, Sharing Rules, Organization-Wide Defaults
Reporting Suite	Salesforce Analytical Reports, Dashboards, Custom LWC Metric Tiles
Testing Framework	Apex Unit Testing Classes (@isTest), Test Data Setup Methods

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The alignment between identified operational challenges and Salesforce architectural capabilities is absolute. Manual quoting bottlenecks are resolved via intuitive Screen Flows; inconsistent rating errors are eradicated through automated Apex calculation handlers; unstructured claim routing is replaced by automated Record-Triggered Flows; and adjuster visibility limitations are overcome through reactive Lightning Web Components.



4.2 Proposed Solution Overview

Design Parameter	Solution Implementation Details
Problem Addressed	Manual, slow, and irregular quoting, issuance, and dispute processing
Proposed Solution	Centralized Salesforce CRM Insurance Policy & Claims Platform
Core Innovation	Automated estimation routines,

	intelligent routing, and multi-tier approvals
Business Impact	Accelerated underwriting, optimized casework, and heightened client satisfaction
Scalability Model	Unified management across Vehicle, Property, and Life lines with flexible data fields
Visibility Standards	Consolidated record views accompanied by specialized adjuster dashboards
Security Framework	Profile-based access and sharing rule enforcement

4.3 Solution Architecture

The multi-line insurance system is constructed upon a robust, 6-tier enterprise architecture:

1. **User Interface Layer:** Direct interaction points for Agents, Adjusters, and Managers via Lightning pages and custom LWCs.
2. **Application & Automation Layer:** Background flows, validation rules, and approval triggers governing business logic execution.
3. **Business Logic Layer:** Apex classes managing complex mathematical calculations and data aggregation wrappers.
4. **Data Layer:** Relational custom objects (Policy__c, Claim__c) mapped to contacts and standard system entities.
5. **Security Layer:** Granular access controls implemented via permission sets and sharing criteria.
6. **Reporting & Visibility Layer:** Analytical dashboards and real-time component views providing clear operational oversight.



5. PROJECT PLANNING & SCHEDULING

- **Methodology:** Agile Software Development Life Cycle (SDLC)
- **Decomposition:** Epics → User Stories → Granular Tasks → Story Point Estimation
- **Execution Cadence:** Two-week sprint iterations with continuous integration and velocity tracking.

Product Backlog, Sprint Schedule, and Estimation

Sprint	Epic / Functional Requirement	Story ID	User Story Description	Story Points	Priority	Team Members
Sprint 1	Data Modelin	US-01	As an admin, I	5	High	Member 1

	g		want to create Policy and Claim objects so that data can be stored systematically.			
Sprint 1	Data Modelin g	US-02	As an admin, I want to configure Vehicle, Property , and Life Record Types for unified management.	3	High	Member 2
Sprint 1	Data Modelin g	US-03	As an admin, I want to build Field Sets to capture product-specific	5	High	Member 3

			attribute s efficientl y.			
Sprint 1	Quoting & Automat ion	US-04	As an agent, I want a guided Auto Quoting Screen Flow to collect data smoothl y.	8	High	Member 4
Sprint 2	Validatio n Rules	US-05	As an agent, I want VIN validatio n during auto quoting to prevent in correc t data entry.	3	High	Member 1
Sprint 2	Apex Program ming	US-06	As the system, I want an Apex Premium Calculat	8	High	Member 2

			or to evaluate rates accurate ly.			
Sprint 2	Flow Integrati on	US-07	As an agent, I want rating logic integrate d with the quoting flow to display estimate s instantly.	5	High	Member 3
Sprint 2	Claims Automat ion	US-08	As a manager , I want claims routed automati cally by policy type to appropri ate support queues.	8	High	Member 4
Sprint 2	Apex Controll	US-09	As an adjuster,	8	High	Member 1

	er		I want a custom controller to retrieve my assigned cases for efficient management.			
Sprint 3	LWC Dashboard	US-10	As an adjuster, I want an interactive dashboard and reusable tiles to monitor caseloads.	8	High	Member 2
Sprint 3	Approval Workflows	US-11	As a manager, I want high-value claims subjected to a structured approval	8	High	Member 3

			process.			
Sprint 4	Security & Testing	US-12	As an admin, I want test classes, sharing rules, and Permission Sets deployed for validation.	8	High	Member 4

Project Tracker, Velocity & Burndown Metrics

Sprint Identifier	Total Story Points	Duration	Planned Start Date	Planned End Date	Points Completed	Actual Release Date
Sprint 1	20	6 Days	24 Oct 2022	29 Oct 2022	20	29 Oct 2022
Sprint 2	20	6 Days	31 Oct 2022	05 Nov 2022	20	05 Nov 2022
Sprint 3	20	6 Days	07 Nov 2022	12 Nov 2022	20	12 Nov 2022
Sprint 4	20	6 Days	14 Nov 2022	19 Nov 2022	20	19 Nov 2022

6. PROJECT DEVELOPMENT PHASE

Learning Objectives:

1. Enterprise Data Modeling & Relational Schema Design
2. Advanced Custom Fields, Relationships, and Field Sets
3. Lightning App Builder Page Composition and Layout Customization
4. Complex Record-Triggered Flow Automation & Invocable Actions
5. Pro-Code Apex Development (Classes, Triggers, Test Coverage)
6. Asynchronous Processing Principles (Batch and Scheduled Apex)
7. Lightning Web Component (LWC) Architecture and Wire Service Integration

Milestone 1: Core Data Model & Policy Configuration

User Story 1: Creation of Custom Objects

1. Create Policy Custom Object:

- Navigate to Setup → Object Manager → Create → Custom Object.
- Configure base properties:
 - **Label:** Policy
 - **Plural Label:** Policies
 - **Record Name:** Policy Number
 - **Data Type:** Auto Number
 - **Display Format:** P-{0000}
 - **Starting Number:** 1
- Enable Optional Features: Allow Reports, Allow Activities, and Track Field History.
- Set Deployment Status to **Deployed** and check **Allow Search**. Click Save.

The screenshot displays the Salesforce Setup interface in the 'Developer Edition'. The navigation menu on the left includes 'Setup', 'Home', and 'Object Manager'. The 'Object Manager' section is expanded, showing a list of objects. The 'Policy' object is selected, and its configuration details are shown on the right. The 'Details' section includes fields for 'Description', 'API Name' (Policy__c), 'Custom' (checked), 'Singular Label' (Policy), 'Plural Label' (Policies), 'Enable Reports' (checked), 'Track Activities' (checked), 'Track Field History' (checked), 'Deployment Status' (Deployed), and 'Help Settings' (Standard Salesforce.com Help Window). The 'Edit' and 'Delete' buttons are visible at the top right of the details section.

2. Define Record Types on Policy Object:

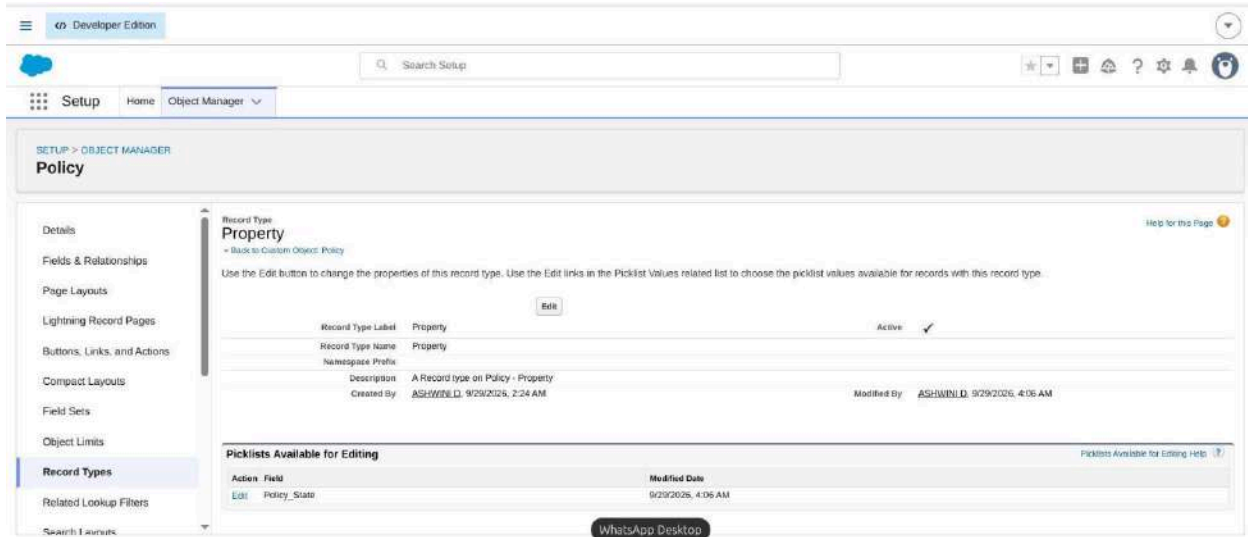
- Navigate to Object Manager → Policy__c → Record Types → New.
- Base on Master, set Record Type Label to Auto, make it active, assign to all profiles, and save.
- Repeat the process to create Property and Life Record Types.

The screenshot shows the Salesforce Object Manager interface for the 'Policy' object. The left sidebar contains navigation links: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, **Record Types**, Related Lookup Filters, and Search Layouts. The main content area displays the 'Auto' record type. It includes a 'Back to Custom Object: Policy' link, an 'Edit' button, and a table with the following data:

Record Type Label	Record Type Name	Namespace Prefix	Description	Created By	Modified By
Auto	Auto		A record type on Policy - Auto	ASHWIN D. 9/29/2025, 2:23 AM	ASHWIN D. 9/29/2025, 4:06 AM

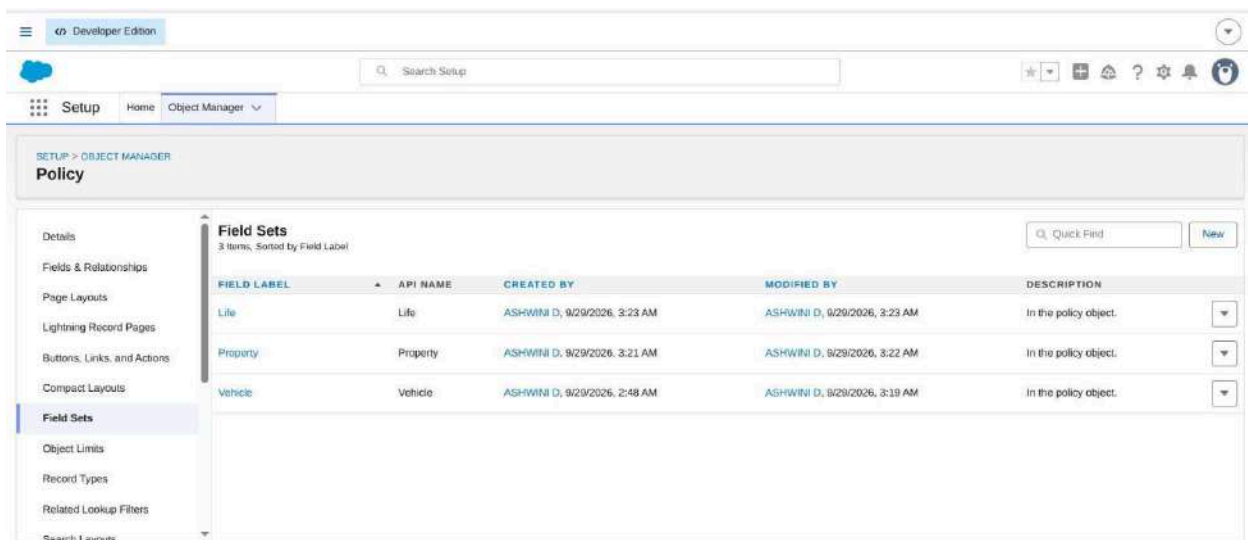
Below the table is a section titled 'Picklists Available for Editing' with a table showing the 'Policy_State' field and its modified date (9/29/2025, 4:06 AM).

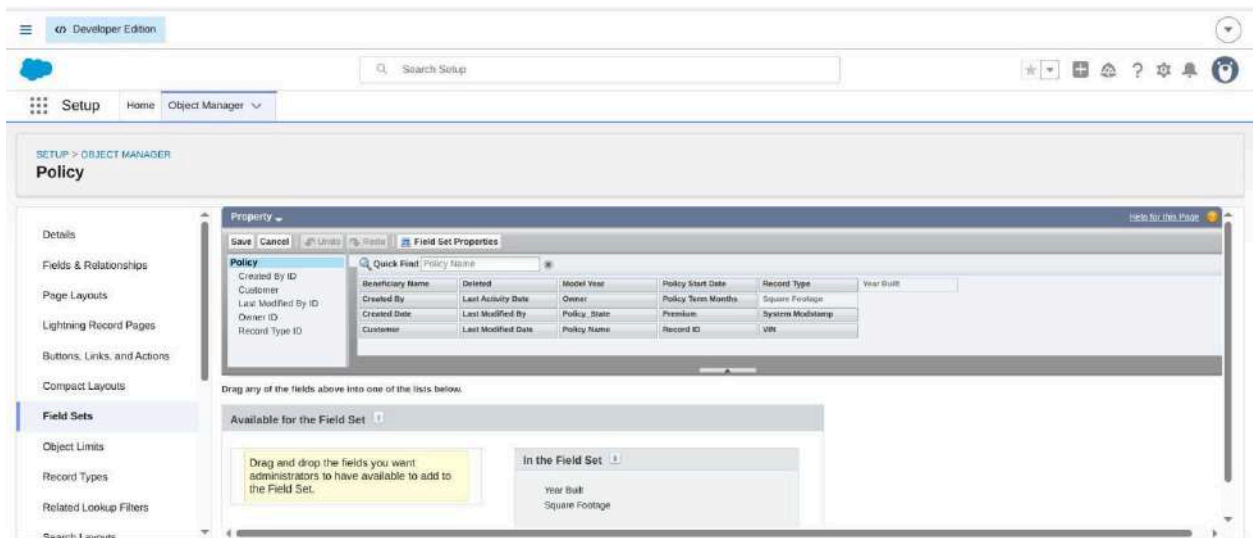
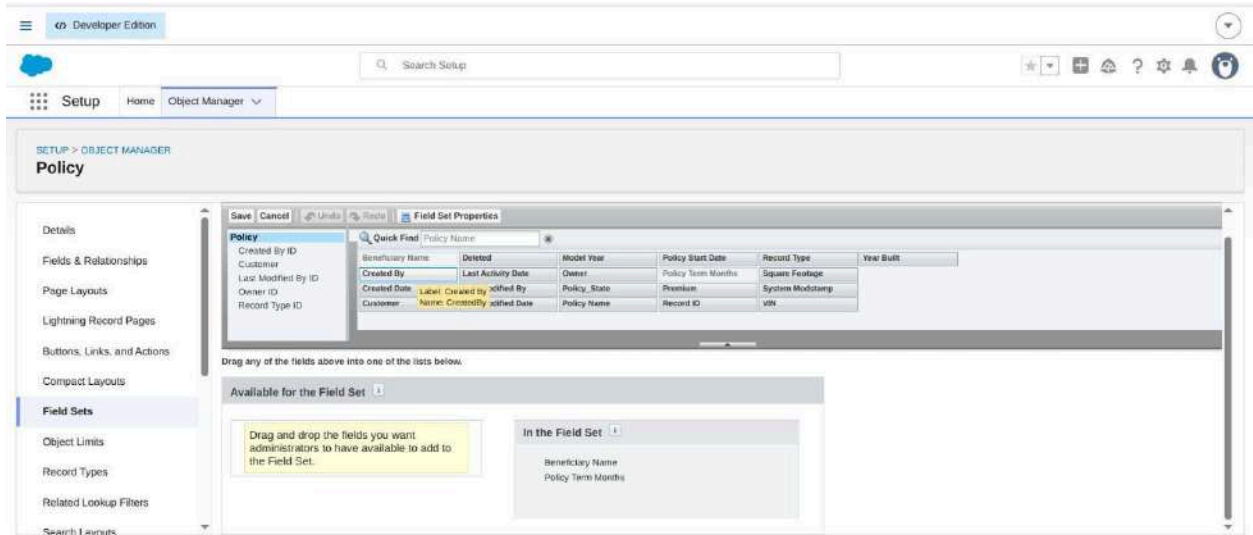
The screenshot shows the Salesforce Object Manager interface for the 'Policy' object, displaying the 'Life' record type. The layout is identical to the previous screenshot, with the 'Record Type Label' set to 'Life' and the 'Record Type Name' set to 'Life'. The 'Description' is 'A Record Type on Policy - Life'. The 'Created By' is 'ASHWIN D. 9/29/2025, 2:28 AM' and the 'Modified By' is 'ASHWIN D. 9/29/2025, 4:06 AM'. The 'Picklists Available for Editing' section also shows the 'Policy_State' field with a modified date of 9/29/2025, 4:06 AM.



3. Create Field Sets for Policy-Specific Data:

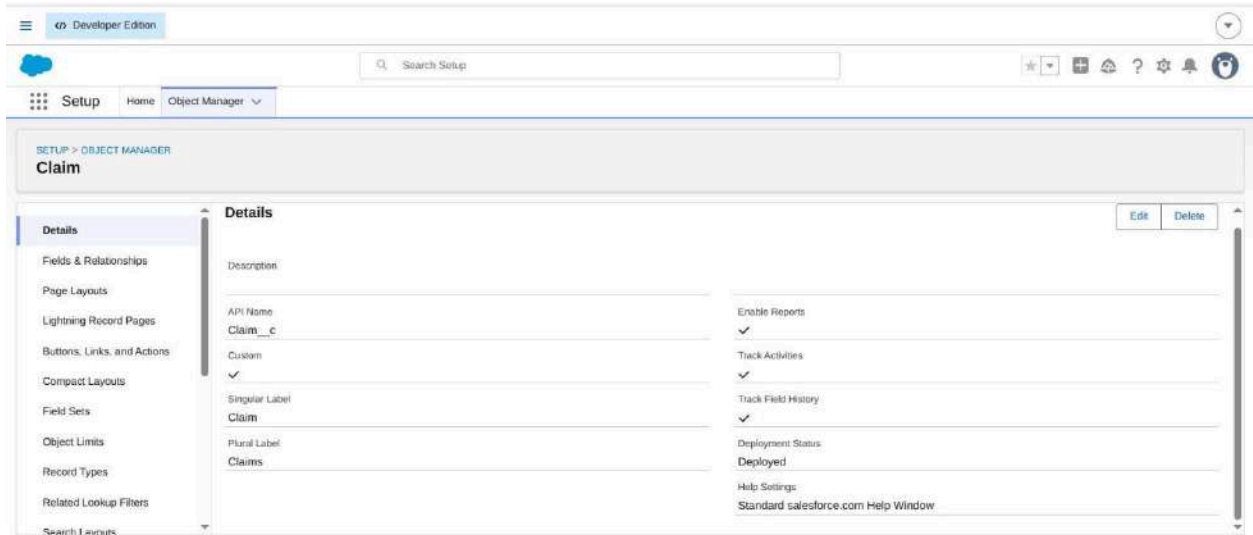
- Navigate to Object Manager → Policy__c → Field Sets → New.
- Create a Field Set labeled Vehicle and add VIN__c and Model_Year__c.
- Repeat to create Property Fields (Square_Footage__c, Year_Built__c) and Life Fields (Beneficiary_Name__c, Policy_Term_Months__c).





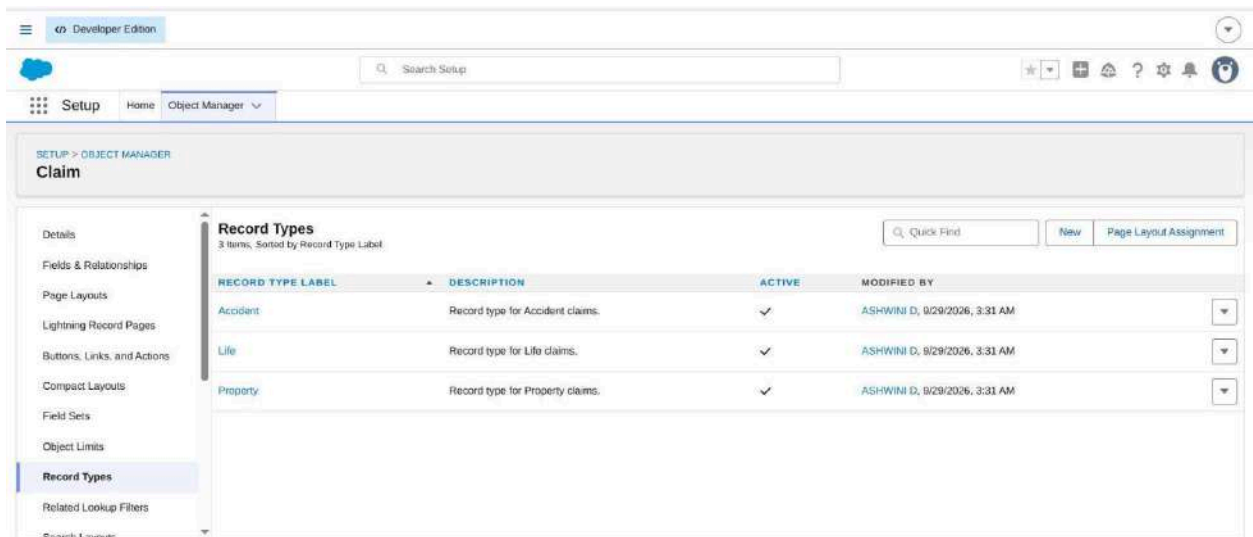
4. Create Fields in the Policy Object:

S.No	Field Label	Data Type	Required Flag
1	Beneficiary Name	Text (50)	False
2	Customer	Lookup (Contact)	False
3	Model Year	Text (5)	False
4	Policy Start Date	Date	False
5	Policy Term	Number (18, 0)	False



6. Define Record Types on Claim Object:

- Create an Accident Record Type on the Claim__c object, make it active, and assign it to all profiles. Repeat for other required types.

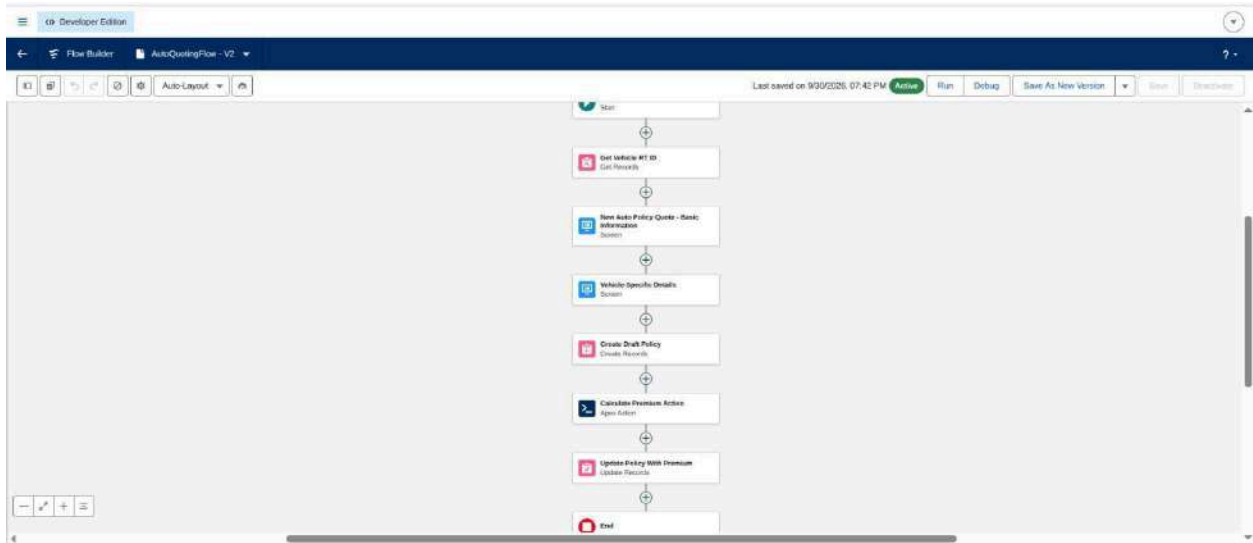


7. Create Fields in the Claim Object:

S.No	Field Label	Data Type	Required Flag
1	Adjuster	Lookup (User)	False
2	Approval Status	Picklist (New, Submitted for Approval, Approved, Rejected)	False
3	Claim Amount	Currency (16, 2)	False
4	Date of Loss	Date/Time	False
5	Description	Long Text Area	False
6	Policy	Lookup (Policy)	False

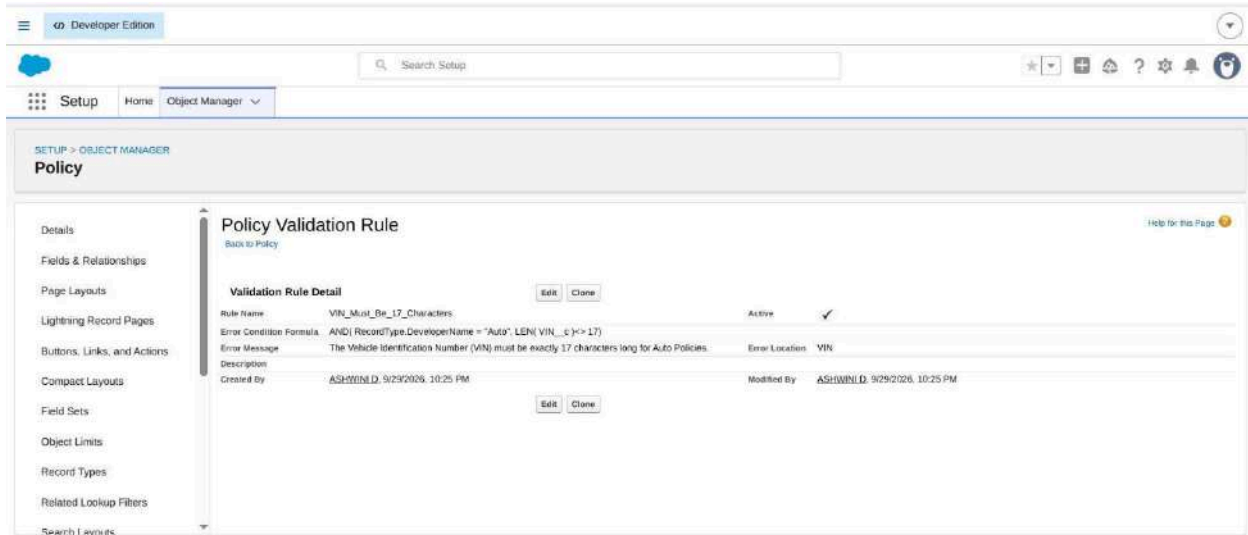
8. Create an AutoQuoting Screen Flow:

- Create a new Screen Flow (AutoQuotingFlow).
- Add a **Get Records** element to retrieve the Auto policy Record Type ID where object equals Policy__c.
- Add a **Screen** element (New Auto Policy Quote - Basic Information) featuring Lookup (Customer__c), Date (Policy Start Date), and Picklist (Policy State) components.
- Add a second **Screen** element (Vehicle-Specific Details) capturing inputs for VIN and Model Year.
- Add a **Create Records** element to establish a draft Policy__c record mapping screen inputs and storing the resulting ID in a text variable (varPolicyId). Activate the flow.



9. Create Validation Rule on Policy Object:

- Create a validation rule named VIN_Must_Be_17_Characters on the Policy__c object.
- **Formula Expression:**
`AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <> 17)`
- Set the error message to specify that the VIN must be exactly 17 characters long for Auto policies, assigning the error location to the VIN__c field.



Milestone 2: Complex Policy Issuance & Claim Routing Automation

1. Create an Apex Class (PremiumCalculator):

```
public class PremiumCalculator {
    public class PremiumInputWrapper {
        @InvocableVariable(required=true)
        public Id policyId;
    }

    @InvocableMethod(label='Calculate Policy Premium' description='Calculates
premium based on policy fields (VIN, Model Year, State).')
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
        List<Decimal> premiumResults = new List<Decimal>();
        Set<Id> policyIds = new Set<Id>();

        for (PremiumInputWrapper input : inputs) {
            if (input.policyId != null) {
                policyIds.add(input.policyId);
            }
        }

        List<Policy__c> policiesToRate = [
            SELECT Id, VIN__c, Model_Year__c, Policy_State__c
```

```

FROM Policy__c
WHERE Id IN :policyIds
];

for (Policy__c policy : policiesToRate) {
    Decimal basePremium = 1000.00;

    if (policy.Policy_State__c == 'CA') {
        basePremium *= 1.15;
    } else if (policy.Policy_State__c == 'TX') {
        basePremium *= 1.05;
    }

    if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018) {
        basePremium += 100;
    }

    premiumResults.add(basePremium);
}
return premiumResults;
}
}

```

The screenshot shows the Salesforce IDE interface. The top pane displays the source code for the `PremiumCalculator` class. The bottom pane shows a log of test results.

Source Code:

```

1 public class PremiumCalculator {
2
3     public class PremiumInputWrapper {
4
5         @InvocableVariable(required=true)
6         public Id policyId;
7
8     }
9
10    @InvocableMethod(
11        label='Calculate Policy Premium'
12        description='Calculates premium based on policy fields (VIN, Model Year, State).'
13    )
14    public static List<Decimal> calculatePremium(
15        List<PremiumInputWrapper> inputs
16    ) {
17
18        List<Decimal> premiumResults = new List<Decimal>();

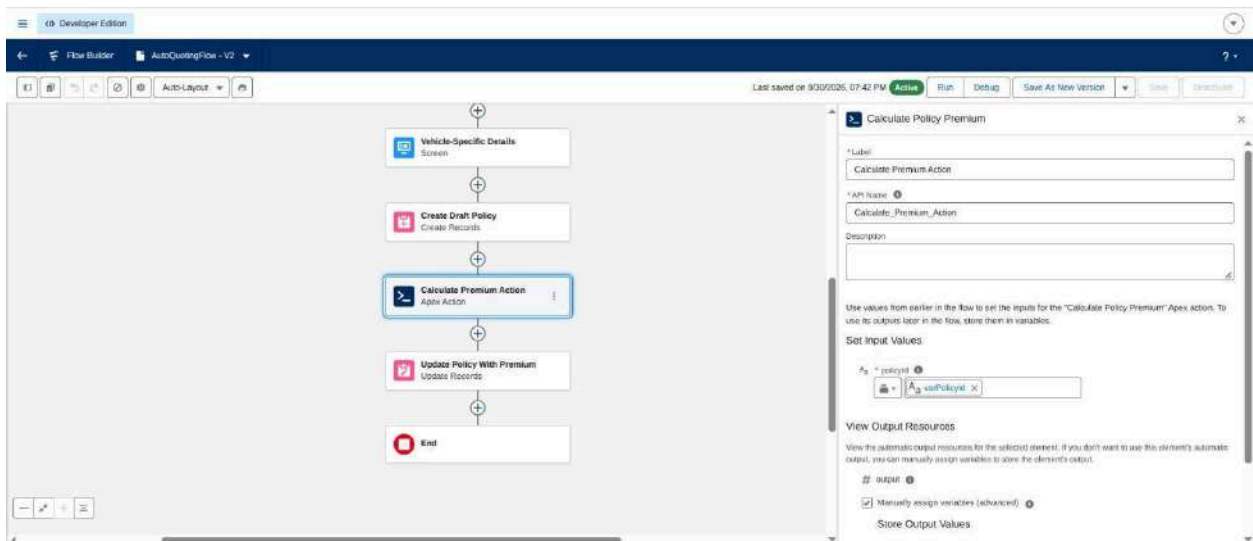
```

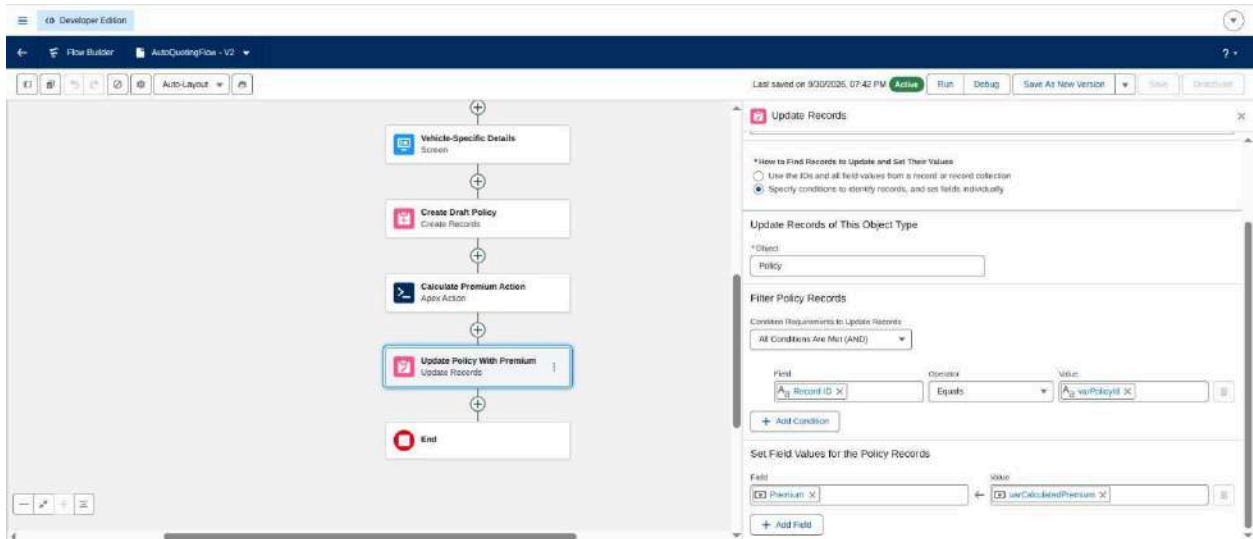
Log:

User	Application	Operation	Time	Status	Read	Size
ASHWIN D	Unknown	ApexTestHandler	10/2/2025, 7:30:36 PM	Success	Unread	2.23 KB
ASHWIN D	Unknown	ApexTestHandler	10/2/2025, 7:30:36 PM	Success	Unread	20.37 KB
ASHWIN D	Unknown	ApexTestHandler	10/2/2025, 7:30:36 PM	Success	Unread	533 bytes

2. Integrate Apex Action into Quoting Flow:

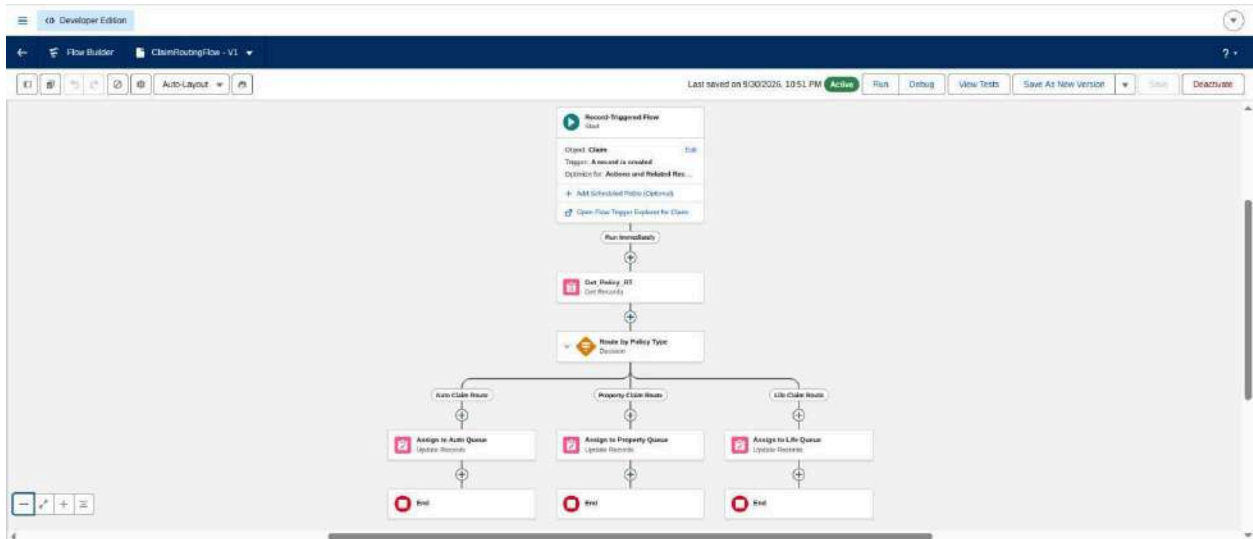
- Open AutoQuotingFlow in Flow Builder.
- Add an **Action** element invoking Calculate Policy Premium, passing varPolicyId as input and mapping the output to a currency variable (varCalculatedPremium).
- Add an **Update Records** element to update the draft policy record with the computed premium amount. Save and activate the new flow version.





3. Create Record-Triggered Flow for Claim Routing:

- Create a new Record-Triggered Flow on the Claim__c object triggered upon record creation.
- Add a **Get Records** element (Get_Policy_RT) to retrieve related policy details and store the Record Type name in varPolicyRTName.
- Add a **Decision** element (Route by Policy Type) with pathways for Auto, Property, and Life.
- Add **Update Records** elements along each branch to assign the appropriate queue ID to the OwnerId field of the triggering claim record.



Milestone 3: Claims Adjuster LWC Dashboard Development

1. Create Apex Controller (ClaimsAdjusterController):

```

public class ClaimsAdjusterController {
    public class ClaimWrapper {
        @AuraEnabled public Id claimId;
        @AuraEnabled public String claimNumber;
        @AuraEnabled public Decimal claimAmount;
        @AuraEnabled public String status;
        @AuraEnabled public String policyType;
        @AuraEnabled public String policyHolderName;
        @AuraEnabled public Integer daysOpen;

        public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
            this.claimId = c.Id;
            this.claimNumber = c.Name;
            this.claimAmount = c.Claim_Amount__c;
            this.status = c.Approval_Status__c;

            Policy__c policyRecord = (Policy__c)p;
            if (policyRecord != null && policyRecord.RecordType != null) {
                this.policyType = policyRecord.RecordType.Name;
            }
        }
    }
}
  
```

```

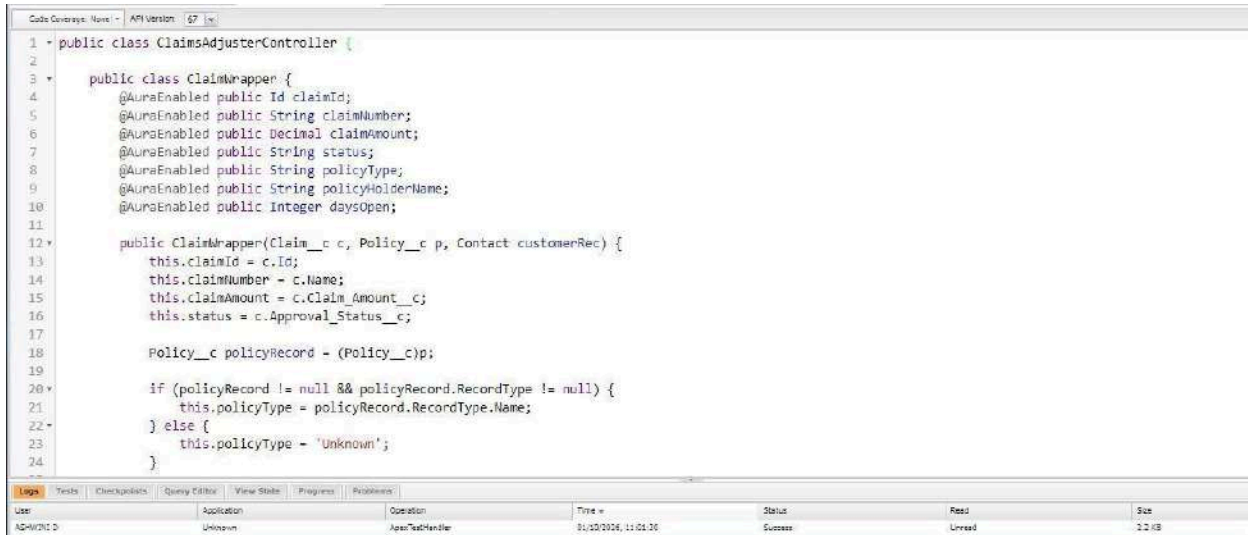
    } else {
        this.policyType = 'Unknown';
    }
    this.policyHolderName = (customerRec != null) ? customerRec.Name : "";
    this.daysOpen = (c.Date_of_Loss__c != null)
        ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
        : 0;
    }
}

```

```

@AuraEnabled(cacheable=true)
public static List<ClaimWrapper> getAssignedClaims() {
    List<ClaimWrapper> results = new List<ClaimWrapper>();
    List<Claim__c> claims = [
        SELECT
            Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
            Policy__r.Name,
            Policy__r.RecordType.DeveloperName,
            Policy__r.Customer__r.FirstName,
            Policy__r.Customer__r.LastName
        FROM Claim__c
        WHERE OwnerId = :UserInfo.getUserId()
        ORDER BY Date_of_Loss__c DESC
    ];
    for (Claim__c claim : claims) {
        Policy__c policy = claim.Policy__r;
        Contact customer = (policy != null && policy.Customer__r != null)
            ? (Contact)policy.Customer__r
            : new Contact();
        results.add(new ClaimWrapper(claim, policy, customer));
    }
    return results;
}
}

```



2. Develop LWC Dashboard Component (claimsDashboardLwc):

- **HTML (claimsDashboardLwc.html):**

```
<template>
```

```
    <lightning-card title="My Assigned Claims Dashboard"
    icon-name="standard:claim">
```

```
        <div class="slds-m-around_medium">
```

```
            <lightning-combobox
```

```
                name="policyFilter"
```

```
                label="Filter by Policy Type"
```

```
                value="All"
```

```
                options={policyTypeOptions}
```

```
                onchange={handleFilterChange}
```

```
                class="slds-m-bottom_medium">
```

```
        </lightning-combobox>
```

```
    <template if:true={visibleClaims}>
```

```
        <div class="slds-grid slds-wrap slds-grid_pull-padded">
```

```
            <template for:each={visibleClaims} for:item="claim">
```

```
                <div key={claim.claimId} class="slds-col slds-size_1-of-1
```

```
slds-medium-size_1-of-2 slds-large-size_1-of-3 slds-p-horizontal_small">
```

```
                <c-claim-tile-lwc claim-data={claim}></c-claim-tile-lwc>
```

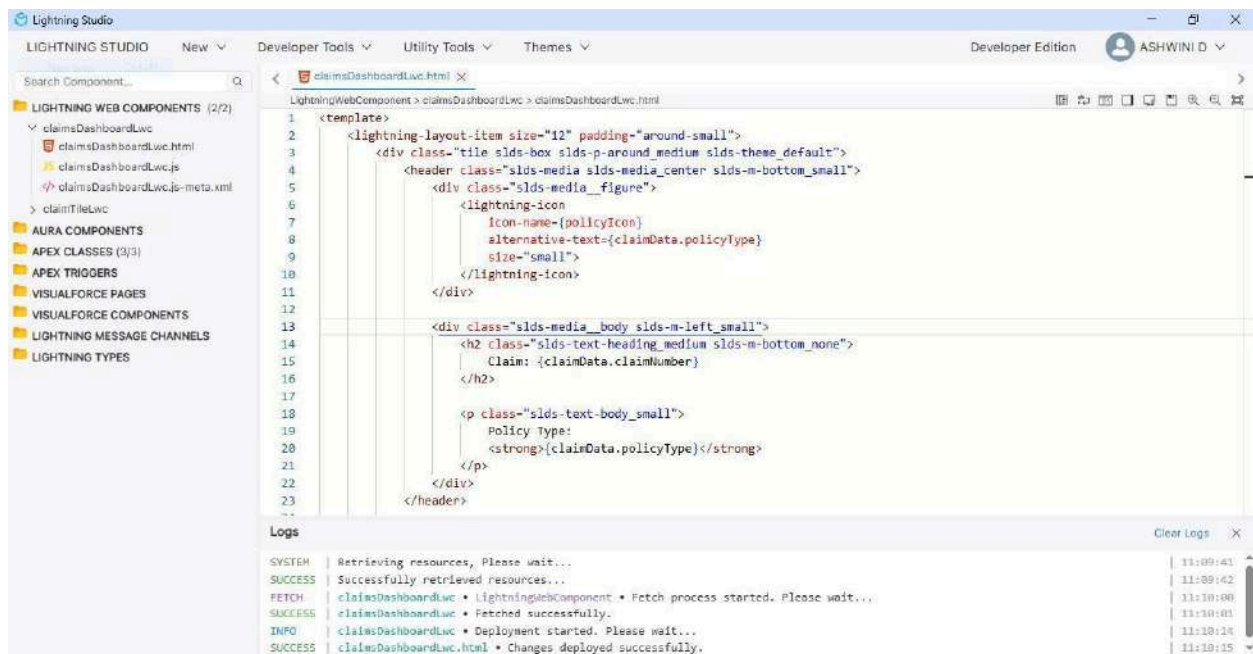
```
            </div>
```

```
        </template>
```

```

    </div>
    <p class="slds-m-top_medium slds-text-heading_small">Total Claims
Assigned: {visibleClaims.length}</p>
  </template>
  <template if:false={visibleClaims}>
    <p>No claims currently assigned to you.</p>
  </template>
</div>
</lightning-card>
</template>

```



- **JavaScript (claimsDashboardLwc.js):**

```

import { LightningElement, wire, track } from 'lwc';
import getAssignedClaims from
 '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

```

```

export default class ClaimsDashboardLwc extends LightningElement {
  @track allClaims = [];
  @track visibleClaims = [];
  error;

```

```

@wire(getAssignedClaims)
wiredClaims({ error, data }) {
  if (data) {
    this.allClaims = data;
    this.visibleClaims = data;
    this.error = undefined;
  } else if (error) {
    this.error = error;
    this.allClaims = undefined;
    this.visibleClaims = undefined;
    console.error('Error retrieving claims:', JSON.stringify(error));
  }
}

handleFilterChange(event) {
  const selectedPolicyType = event.target.value;
  if (selectedPolicyType === 'All') {
    this.visibleClaims = this.allClaims;
  } else {
    this.visibleClaims = this.allClaims.filter(claim =>
      claim.policyType === selectedPolicyType
    );
  }
}

get policyTypeOptions() {
  return [
    { label: 'All Policy Types', value: 'All' },
    { label: 'Auto', value: 'Auto' },
    { label: 'Property', value: 'Property' },
    { label: 'Life', value: 'Life' }
  ];
}

```

- **Configuration Metadata (claimsDashboardLwc.js-meta.xml):**

```

<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__HomePage</target>
  </targets>
</LightningComponentBundle>

```

3. Develop Reusable Tile Component (claimTileLwc):

- **HTML (claimTileLwc.html):**

```

<template>
  <lightning-layout-item size="12" padding="around-small">
    <div class="tile slds-box slds-p-around_medium slds-theme_default">
      <header class="slds-media slds-media_center slds-m-bottom_small">
        <div class="slds-media__figure">
          <lightning-icon icon-name={policyIcon}
alternative-text={claimData.policyType} size="small"></lightning-icon>
        </div>
        <div class="slds-media__body slds-m-left_small">
          <h2 class="slds-text-heading_medium slds-m-bottom_none">
            Claim: {claimData.claimNumber}
          </h2>
          <p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>
        </div>
      </header>
      <div class="slds-border_top slds-p-top_small">
        <p>Policy Holder: <strong>{claimData.policyHolderName}</strong></p>
        <lightning-formatted-number
          value={claimData.claimAmount}
          format-style="currency"
          currency-code="USD"
          label="Claim Amount">
        </lightning-formatted-number>
      </div>
    </div>
  </lightning-layout-item>
</template>

```

```

        <p>Status: {claimData.status}</p>
        <p class="slds-text-body_regular">
            Days Open: <strong
class="slds-text-color_error">{claimData.daysOpen}</strong>
        </p>
    </div>
</div>
</lightning-layout-item>
</template>

```

- **JavaScript (claimTileLwc.js):**

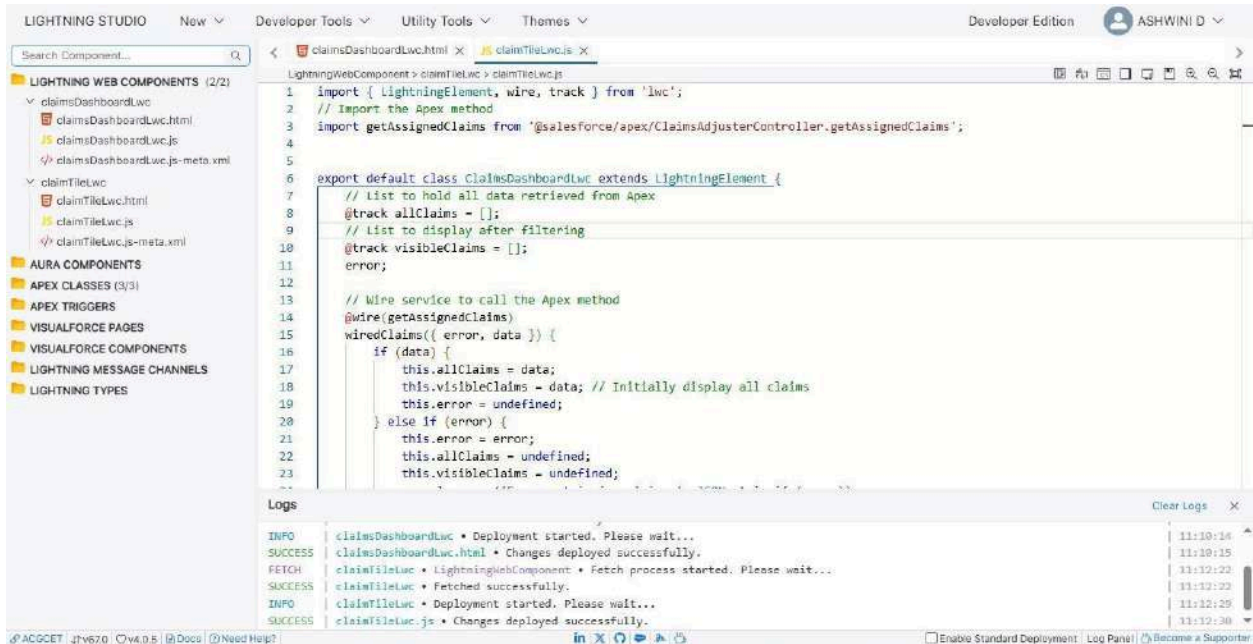
```

import { LightningElement, api } from 'lwc';

export default class ClaimTileLwc extends LightningElement {
    @api claimData;

    get policyIcon() {
        if (this.claimData && this.claimData.policyType) {
            const type = this.claimData.policyType.toLowerCase();
            if (type === 'auto') return 'utility:truck';
            if (type === 'property') return 'utility:home';
            if (type === 'life') return 'utility:connected_apps';
        }
        return 'utility:help';
    }
}

```



- **Configuration Metadata (claimTileLwc.js-meta.xml):**

```
<?xml version="1.0"?>
```

```
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
```

```
  <apiVersion>63.0</apiVersion>
```

```
  <isExposed>true</isExposed>
```

```
</LightningComponentBundle>
```

Milestone 4: Advanced Claim Processing, Security & Code Setup

1. Add Approval Status Field to Claim Object:

- Navigate to Object Manager → Claim__c → Fields & Relationships → New.
- Select Picklist data type, label as Approval Status, enter values (New, Submitted for Approval, Approved, Rejected), and save.

SETUP > OBJECT MANAGER

Claim

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Claim Custom Field: **Approval Status**
Back to Claim

Custom Field Definition Detail

Field Information

Field Label	Approval Status	Object Name	Claim
Field Name	Approval_Status	Field Type	Picklist
API Name	Approval_Status__c		
Description			
Help Text			
Data Owner			
Field Usage			
Data Security Level			
Compliance Categorization			
Created by	ASHWIN D. 6/29/2020, 2:31 AM	Modified by	ASHWIN D. 6/29/2020, 2:31 AM

General Options

Required ☐

Default Value

Picklist Options

Restrict picklist to the values defined in the value set ☒

Controlling Field [New](#)

Picklist Values Used

Developer Edition

Manage Approvals

Search Setup

Setup

Home

Object Manager

SETUP > OBJECT MANAGER

Claim

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Controlling Field [New](#)

Picklist Values Used

Active and Inactive picklist values: 4 (1,000 max)

Field Dependencies

No dependencies defined.

Validation Rules

No validation rules defined.

Values

Action	Value	API Name	Default	Chart Colors	Created By
☐ Edit Del Deactivate	New	New	☐	Assigned dynamically	ASHWIN D. 6/29/2020, 2:31 AM
☐ Edit Del Deactivate	Submitted for Approval	Submitted for Approval	☐	Assigned dynamically	ASHWIN D. 6/29/2020, 2:31 AM
☐ Edit Del Deactivate	Approved	Approved	☐	Assigned dynamically	ASHWIN D. 6/29/2020, 2:31 AM
☐ Edit Del Deactivate	Rejected	Rejected	☐	Assigned dynamically	ASHWIN D. 6/29/2020, 2:31 AM

Inactive Values

No inactive values defined.

Always show me more records per related list

2. Configure Approval Process (High Value Claim Approval):

- Navigate to Setup → Approval Processes → Manage Approval Processes for Claims.
- Create a new approval process using the standard setup wizard named High Value Claim Approval.

- Set entry criteria: Claim Amount > 50000.
- Configure two approval steps:
 - **Step 1:** Assigned automatically to the Senior Adjuster user.
 - **Step 2:** Assigned automatically to the Department Manager user.
- Configure initial submission, final approval, and rejection field updates to manage the Approval Status picklist field. Activate the process.

The screenshot displays the Salesforce Setup interface for configuring an approval process. The left sidebar shows the navigation menu with 'Setup' selected. The main content area is titled 'Approval Processes' and shows the configuration for a process named 'Claim: High Value Claim Approval'.

Process Definition Detail

Process Name	High Value Claim Approval	Active	☒
Unique Name	High_Value_Claim_Approval	Next Automated Approver Determined By	
Description			
Entry Criteria	CLAMT: CLAMT: Amount exceeds more 50000		
Initial Submission	Administrator ONLY	Allow Submission to Recal Approval Requests	☐
Approval Assignment Email Template	Claim Count		
Created By	ASHWIN D.	Created On	3/30/2025, 11:09 AM
Modified By	ASHWIN D.	Modified On	3/30/2025, 12:08 PM

Initial Submission Actions

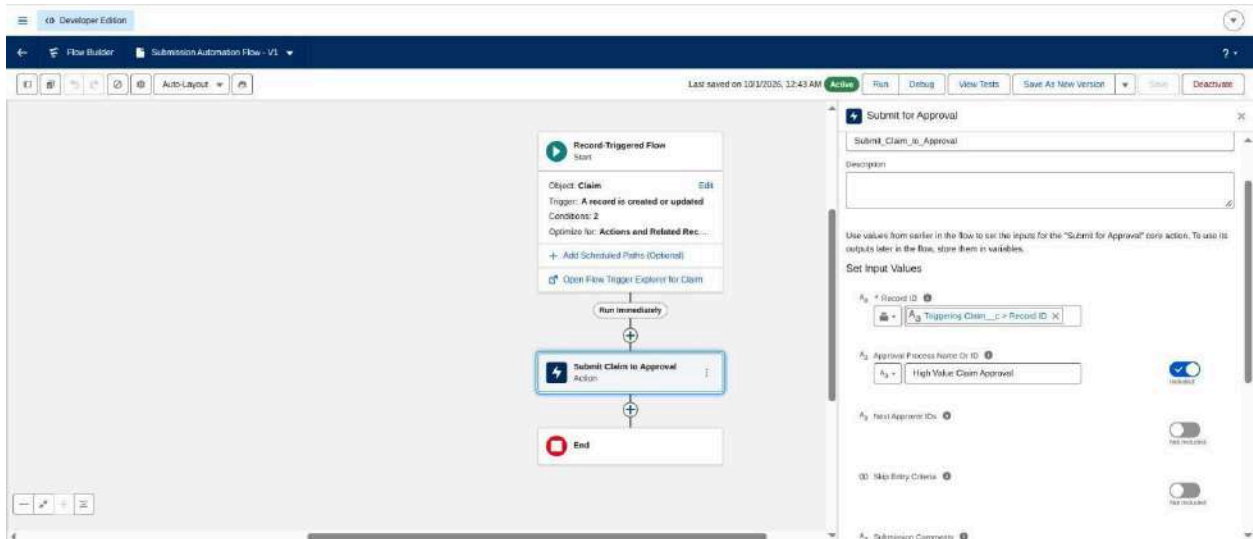
Action	Type	Description
Record Lock	Field Update	Lock the record from being edited
Field Update	Field Update	Update Status Submitted

Approval Steps

Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Show Actions Edit	1	Step 1: Senior Adjuster			User: Senior Adjuster	Final Rejection
Show Actions Edit	2	Step 2: Manager Review			User: Department Manager	Final Rejection

3. Create Submission Automation Flow:

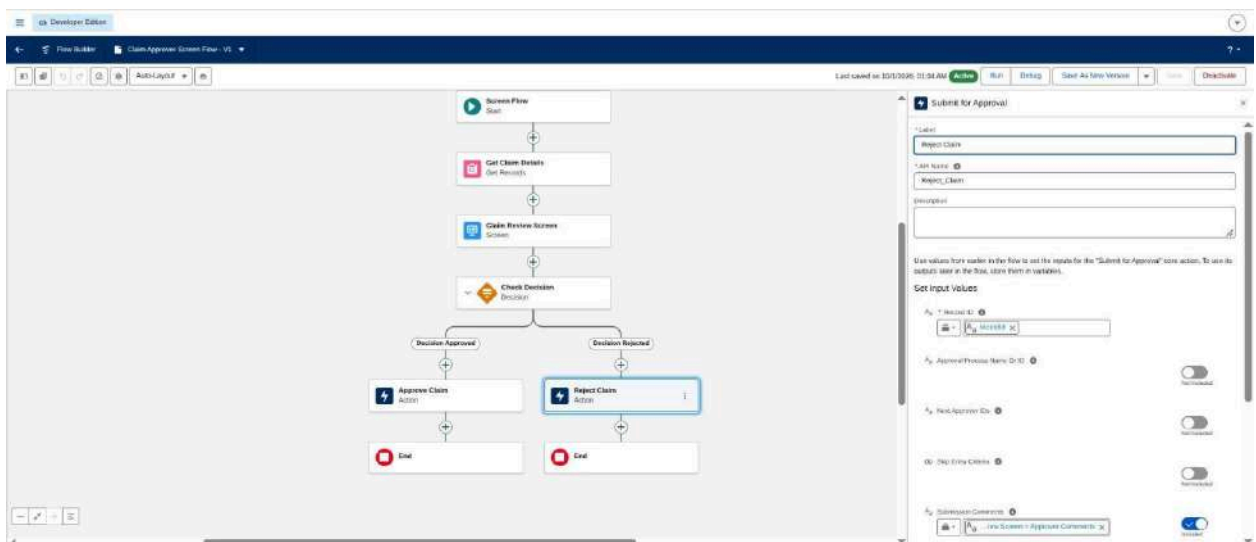
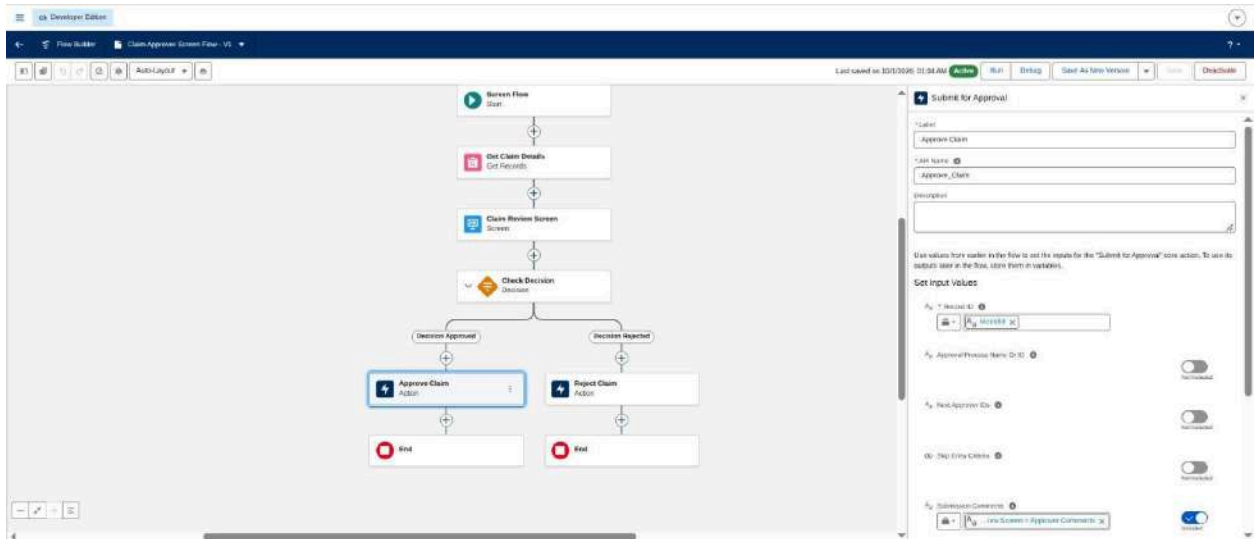
- Create a Record-Triggered Flow on Claim__c executing when created or updated where Claim Amount > 50000 and Approval Status equals New.
- Add an **Action** element to submit the record to the High Value Claim Approval process. Save and activate.



4. Implement Reviewer Screen Flow (Claim Approver Screen Flow):

- Create a Screen Flow with an input variable named recordId.
- Add a **Get Records** element (Get_Claim_Details) to fetch claim data where ID equals recordId.
- Add a **Screen** element (Claim Review Screen) displaying summary text, radio buttons for decision (Approve or Reject), and a long text area for comments.
- Add a **Decision** element (Check Decision) directing execution to specialized **Action** elements that process approvals or rejections. Save and activate.





Create Quick Action & Page Layout Assignment:

- Create a new Quick Action on the Claim__c object pointing to the Claim Approver Screen Flow, labeled as Approve/Reject Claim.
- Add the action to the Salesforce Mobile and Lightning Experience Actions section on the Claim page layout.

5. Create Apex Test Class (ClaimsAdjusterControllerTest):

```

@isTest
private class ClaimsAdjusterControllerTest {
    @testSetup
    static void setupTestData() {
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
        User u = new User(
            FirstName = 'Test',
            LastName = 'User',
            Alias = 'tuser',
            Username = 'testuser1234343@example.com.test',
            Email = 'testuser123@example.com',
            TimeZoneSidKey = 'America/Los_Angeles',
            LocaleSidKey = 'en_US',
            EmailEncodingKey = 'UTF-8',
            LanguageLocaleKey = 'en_US',
            ProfileId = p.Id
        );
        insert u;

        Id policyRT =
        Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName().get('Auto').get
        RecordTypeId();
        Id claimRT =
        Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName().get('Accident')
        .getRecordTypeId();

        Contact con = new Contact(
            FirstName = 'John',
            LastName = 'Doe',
            OwnerId = u.Id
        );
        insert con;

        Policy__c policy = new Policy__c(
            Customer__c = con.Id,
            RecordTypeId = policyRT,
            OwnerId = u.Id,

```

```

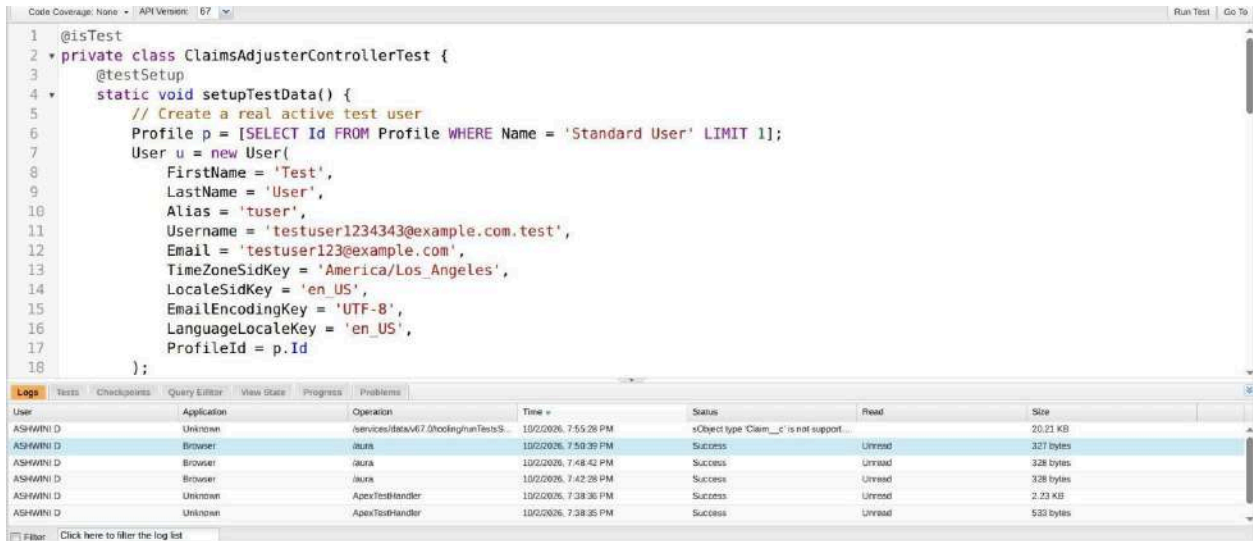
        VIN__c = '1A903284375893532'
    );
    insert policy;

    Claim__c claim = new Claim__c(
        Policy__c = policy.Id,
        RecordTypeId = claimRT,
        Claim_Amount__c = 5000,
        Date_of_Loss__c = Date.today().addDays(-10),
        OwnerId = u.Id
    );
    insert claim;
}

@Test
static void testGetAssignedClaims() {
    User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
    System.runAs(u) {
        Test.startTest();
        List<ClaimsAdjusterController.ClaimWrapper> claims =
ClaimsAdjusterController.getAssignedClaims();
        Test.stopTest();

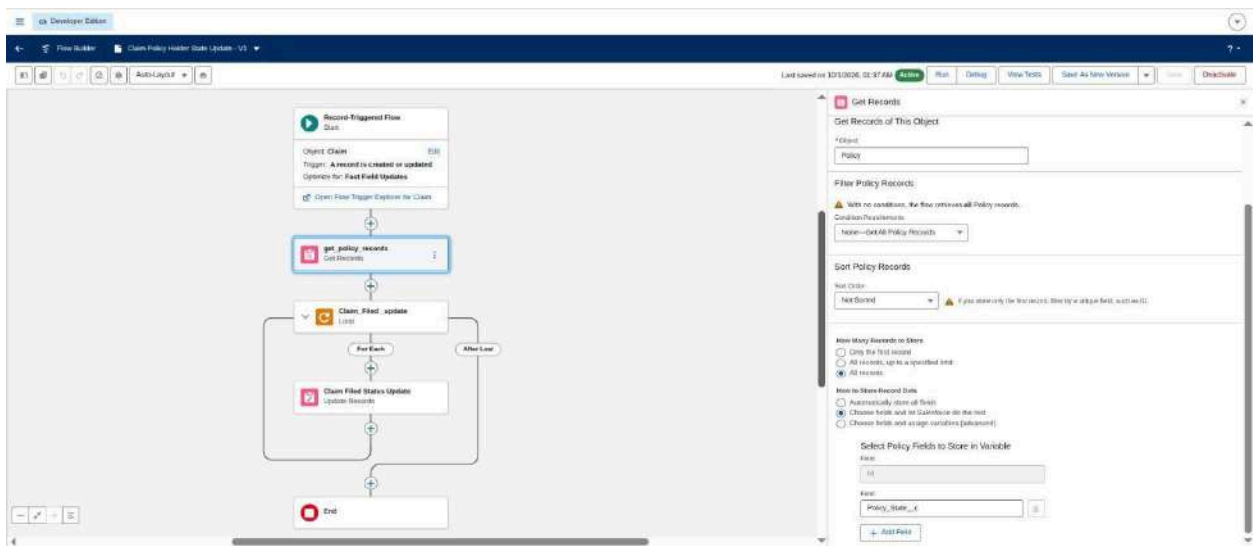
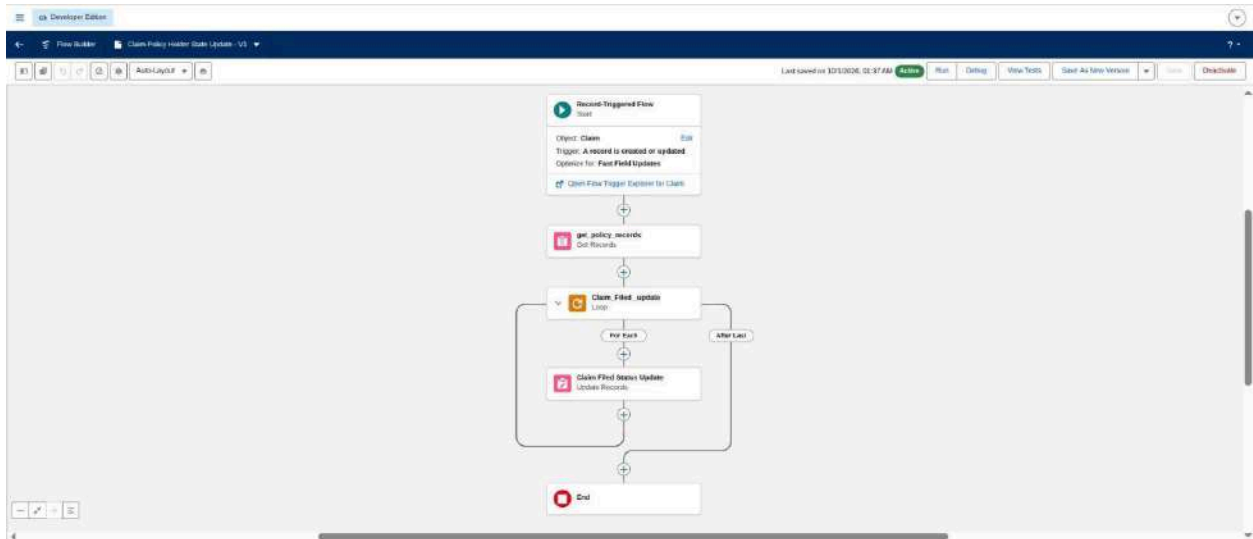
        System.assertNotEquals(null, claims, 'Claims list should not be null');
    }
}
}

```



6. Refine the Sharing Rules to ensure Adjusters only see claims in their assigned territory/state

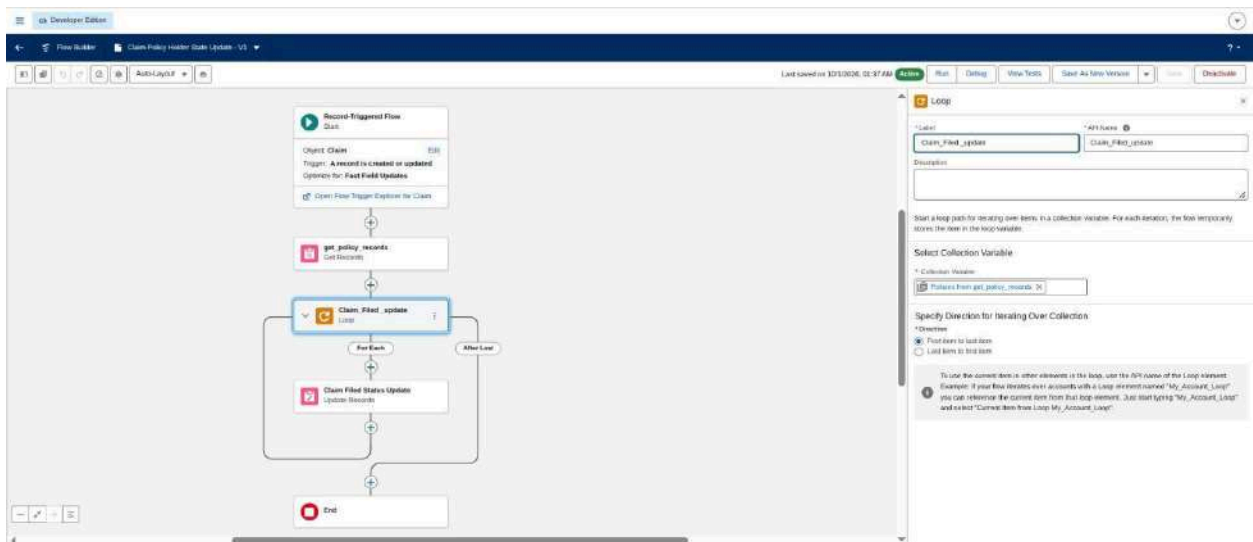
- Navigate to Setup, type "Flows" into the Quick Find box, and open Flows.
- Click on New Flow, select Record-Triggered Flow, choose the Claim object, and configure it to trigger when a record is created or updated.
- Set entry conditions to None and optimize the flow for Updates.
- Add a Get Records element to the canvas named `get_policy_records`, set the filter criteria to None, store all records, and choose fields manually while selecting the Policy State field.
- Add a Loop element named `Claim_Filed_update` to iterate over the collection of policies from the Get Records element from first to last item.
- Inside the loop, add an Update Records element named `Claim_Filed_state_update` with filter conditions set to None, mapping the policy state from the current loop to the Policy Account Holder State field.
- Save and activate the flow under the name Claim Policy Holder State Update.



6.1 Define Sharing Rules for Claim Object

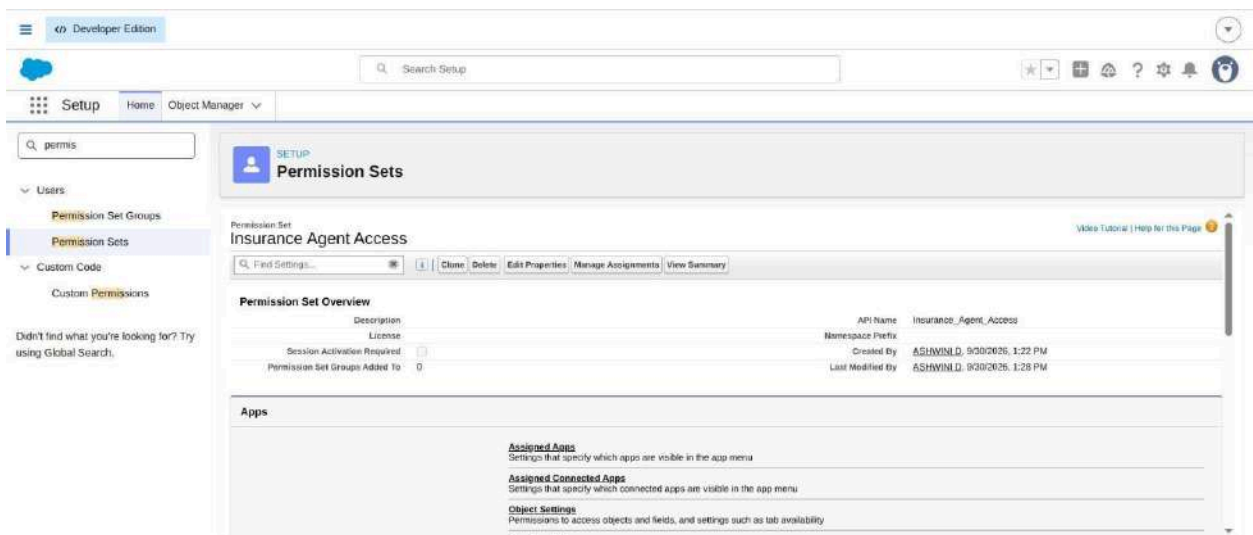
- In Setup, search for Sharing Settings, open the menu, select the Claim object under "Manage Sharing Settings for", and scroll to Claim Sharing Rules.
DOCX
- Click New, label the rule as **Share Records based on Policy State CA**, set the criteria so that Policy Account Holder State Equals CA, grant Read/Write access, and save.
DOCX
- Create a second sharing rule labeled **Share Records based on Policy**

State with the criteria set to Policy Account Holder State Equals TX, assign Read/Write access, and save.
DOCX



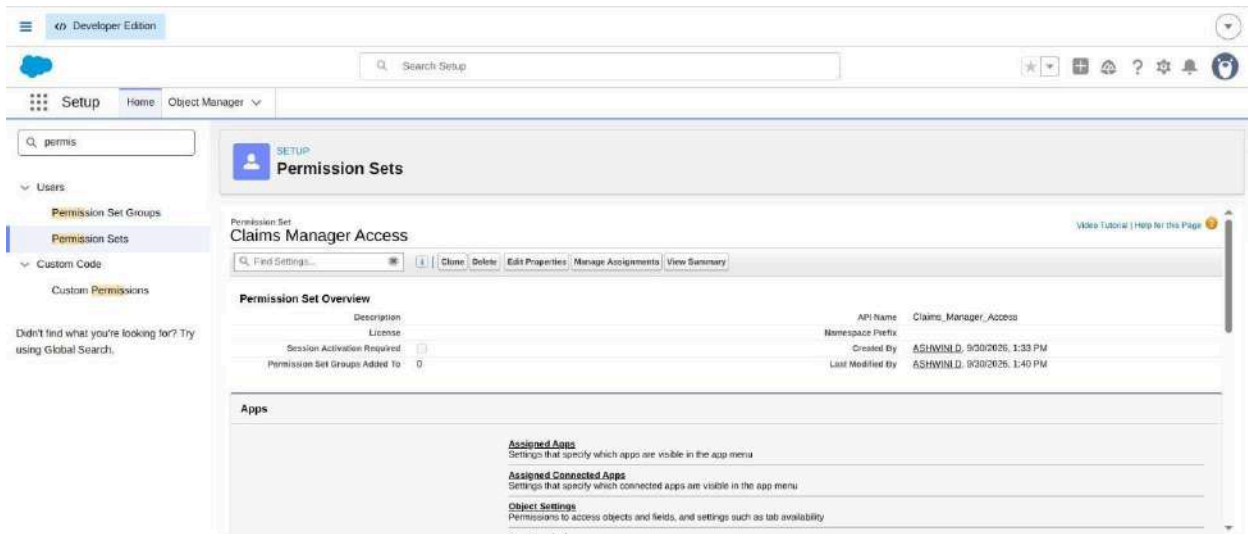
7. Finalize all Permission Sets to differentiate access between Agents, Adjusters, and Managers

- Go to Setup, search for Permission Sets, and click New.
- Assign the Label **Insurance Agent Access**, grant Create/Read access for the Policy object, Read/Edit for the Contact object, and Read Only for the Claim object.
- Enable Flow User under System permissions and authorize the **PremiumCalculator** Apex class.



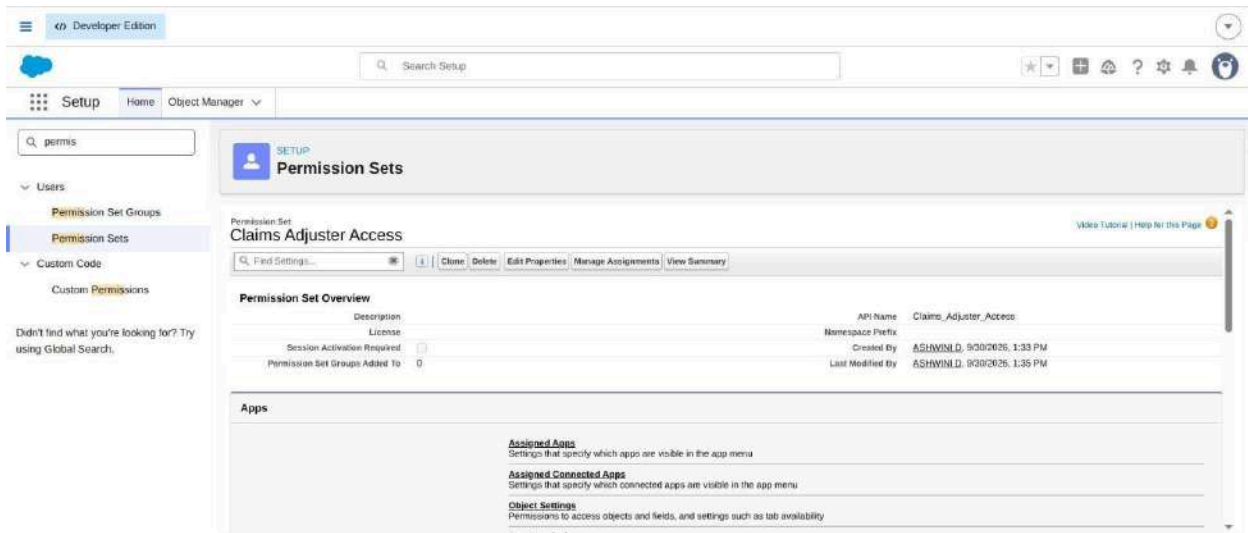
7.1 Create Claims Manager Access Permission Set

- Click New and label the set **Claims Manager Access**.
- Provide Read Only access for the Policy object, Read/Edit for the Contact object, and Read/Edit/Delete for the Claim object.
- Enable Run Reports and Manage Approvals under System permissions.



7.2 Create Claims Adjuster Access Permission Set

- Click New and label the set **Claims Adjuster Access**.
- Grant Read Only access for the Policy object and Read/Edit access for the Claim object.
- Enable the **ClaimsAdjusterController** under Apex Classes.



7. FUNCTIONAL & PERFORMANCE TESTING

During testing phases, validation captures were taken directly from the Salesforce environment to verify vehicle registration workflows, dealer allocation matrices, inventory validation rules, status refresh triggers, batch executions, automated notifications, and executive dashboard metrics. All features passed functional benchmarks to guarantee optimal performance under standard operational loads.

8. ADVANTAGES & DISADVANTAGES

Advantages

- **Centralized Platform:** Consolidates Vehicle, Property, and Life lines into a unified Salesforce database architecture.
- **Automated Quoting:** Guided Screen Flows greatly simplify the policy issuance pipeline.
- **Automated Premium Calculation:** Apex engines consistently compute rate structures without manual lag.
- **Automated Claim Routing:** Dynamic rules route inbound claims instantly to designated support queues.
- **Accelerated Processing:** Automated approvals and tracking mechanisms eliminate workflow bottlenecks.
- **Enhanced Visibility:** The custom LWC Claims Dashboard grants adjusters a focused, real-time workload overview.
- **Data Integrity:** Stringent validation guardrails secure structural accuracy across

data fields.

- **Flexible Architecture:** Modular Record Types and Field Sets adapt seamlessly to diverse product requirements.
- **Secured Accessibility:** Customized Permission Sets and Sharing Rules protect sensitive geographic data.
- **High Scalability:** The system architecture naturally extends to support future insurance lines and features.

Disadvantages

- **Complex Data Modeling:** Diverse line requirements demand extensive data relationship tuning.
- **Approval Multiplicity:** Multi-tier approvals require careful coordination across adjustment tiers.
- **User Adoption Curve:** Personnel require targeted training on new interactive screen flows to maximize efficiency.
- **Integration Overhead:** External verification mocks and API handlers demand specialized Apex unit test coverage.
- **Platform Dependency:** Operations rely heavily on Salesforce ecosystem uptime and governor limits.
- **Implementation Effort:** Configuring complex flows, components, permissions, and validation structures requires deep technical maintenance.

9. CONCLUSION

The Multi-Line Insurance Policy and Claims Management System delivers a robust, consolidated Salesforce framework that modernizes traditional insurance operations. By replacing disjointed manual procedures with intelligent automated workflows, dynamic calculations, structural approvals, and reactive LWC dashboards, the system resolves historical throughput bottlenecks.

Ultimately, the deployment proves how native platform components—including Flows, Apex controllers, Record Types, and Sharing Rules—can unify disparate product portfolios onto a single scalable ecosystem, elevating both internal staff efficiency and end-customer service quality.

10. FUTURE SCOPE

The capabilities of the platform can be further scaled through several upcoming initiatives:

- **External Integrations:** Direct connection with live third-party vehicle and

property registry databases for instant data verification.

- **Expanded Line Offerings:** Support for commercial, travel, and specialty insurance categories beyond the foundational three lines.
- **Advanced Automation:** Deeper algorithmic claim evaluation and autonomous risk scoring models.
- **Comprehensive Analytics:** Expanded executive reporting matrices for granular profitability and claims trend analysis.
- **Customer Portal Integration:** Self-service capabilities via Experience Cloud allowing policyholders to check status updates independently.
- **Mobile Field Access:** Mobile-optimized layouts empowering field agents and adjusters to manage workflows on the go.
- **AI-Powered Assistance:** Implementation of generative AI models to draft adjuster notes and accelerate customer interactions.
- **API Architecture Expansion:** Broadened integration webhooks connecting legacy accounting and third-party payment gateways.

11. APPENDIX

- **Objects:** Policy, Claim, Contact, User
- **Record Types:** Vehicle, Property, Life; Accident, Property, Life
- **Automation:** Screen Flows, Record-Triggered Flows, Approval Processes
- **Apex Controllers:** `PremiumCalculator`, `ClaimsAdjusterController`
- **Lightning Web Components (LWC):** Claims Dashboard, Claim Tile
- **Validation Rules:** 17-character VIN verification constraint
- **Security Framework:** Permission Sets, State-Based Criteria Sharing Rules
- **Testing Suite:** `ClaimsAdjusterControllerTest`
- **Reporting Architecture:** Custom LWC Dashboard & standard Salesforce reporting matrices